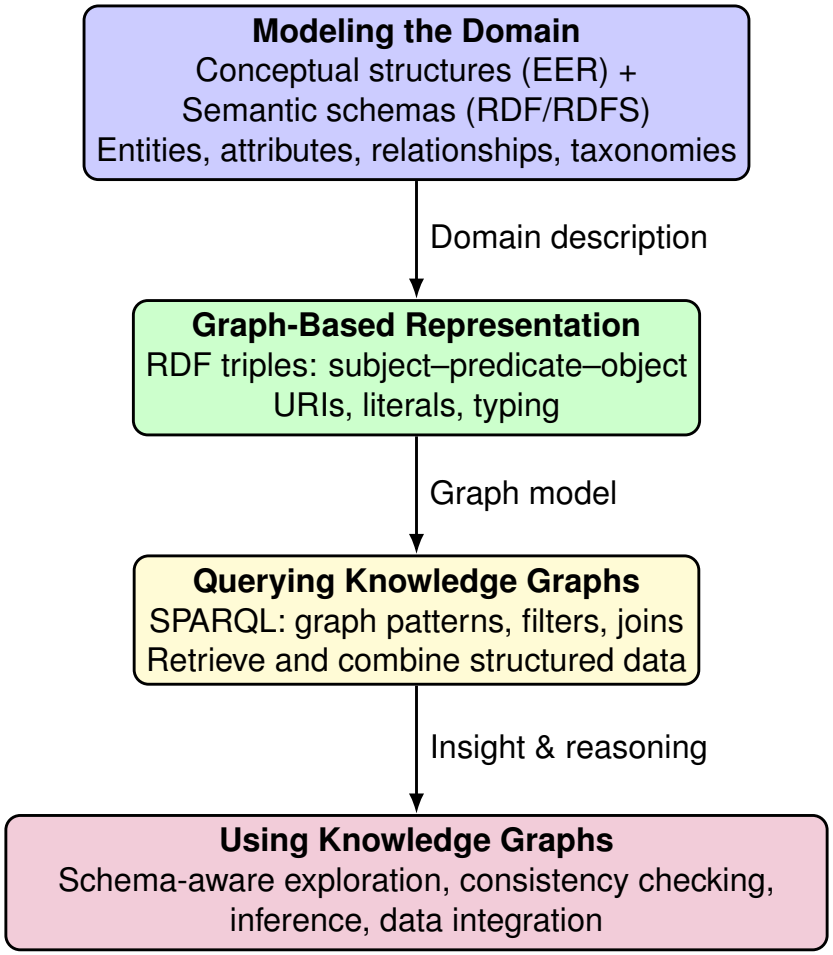
Semantic Data Modeling (Lecture 4)

From Data to Knowledge

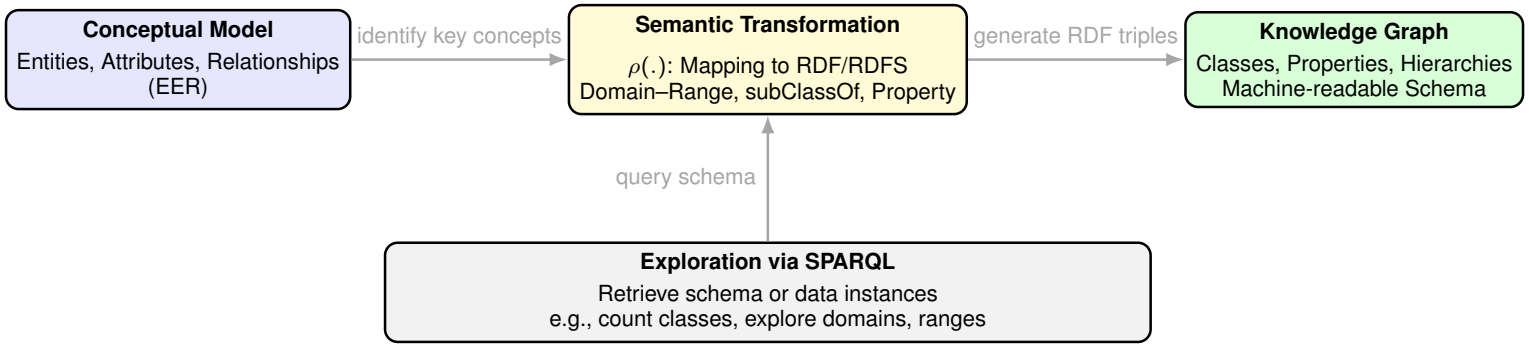


From understanding the domain → representing it formally → querying it → adding semantics.

Semantic Data Modeling

From Conceptual Design to Semantic Models

Semantic Data Modeling bridges the gap between conceptualization and machine-interpretable representations by mapping **entities, attributes, and relationships** into **knowledge graphs**.



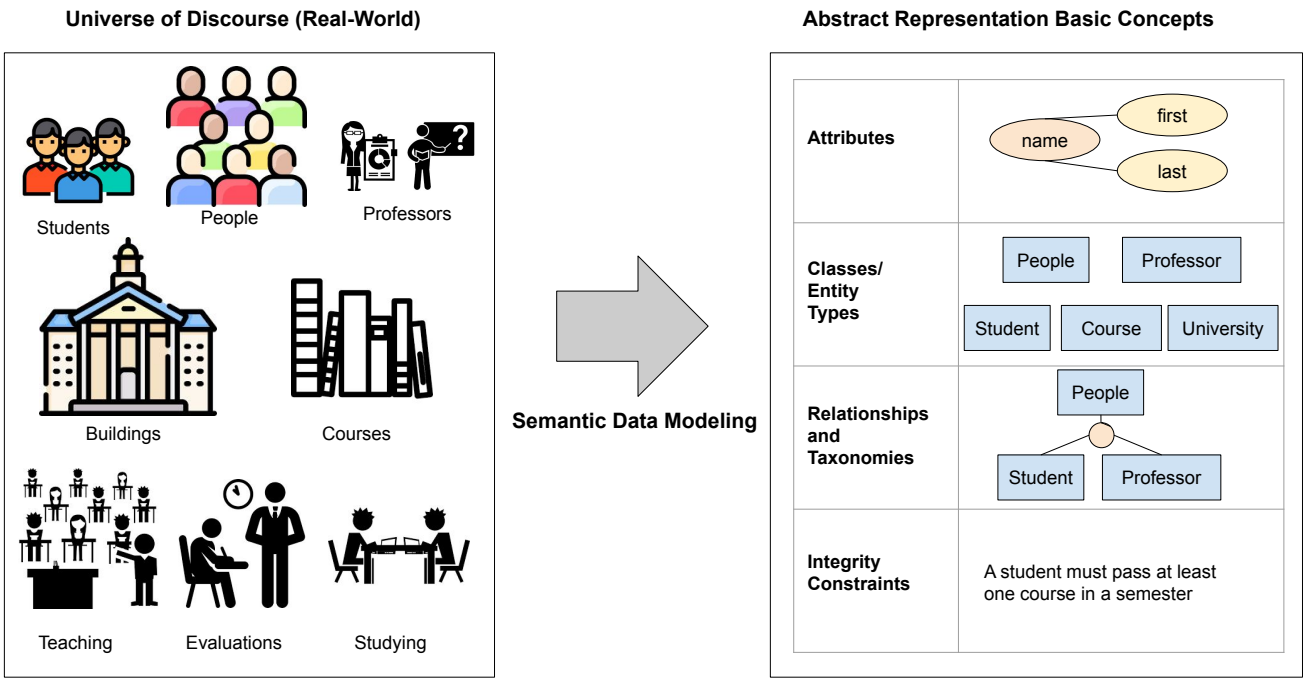
Semantic Data Modeling = conceptual understanding + formal representation + queryability.

Agenda

- Basic Concepts of Semantic Data Modeling
  - Attributes
  - Entities
  - Relationships
  - Classification (disjoint and overlapped)
- Typical mistakes in the representation of basic concepts
- Representing Basic Concepts in RDF and RDFS
- Exploring RDFS graphs

Semantic Data Modeling [1] (1/2)

- **Definition:** Identifies and represents key concepts relevant to a universe of discourse.
- **Purpose:**
  - Capture and express knowledge that characterizes key concepts.
  - Enable representation in a knowledge base.



Semantic Data Modeling [1] (2/2)

Basic Concepts:

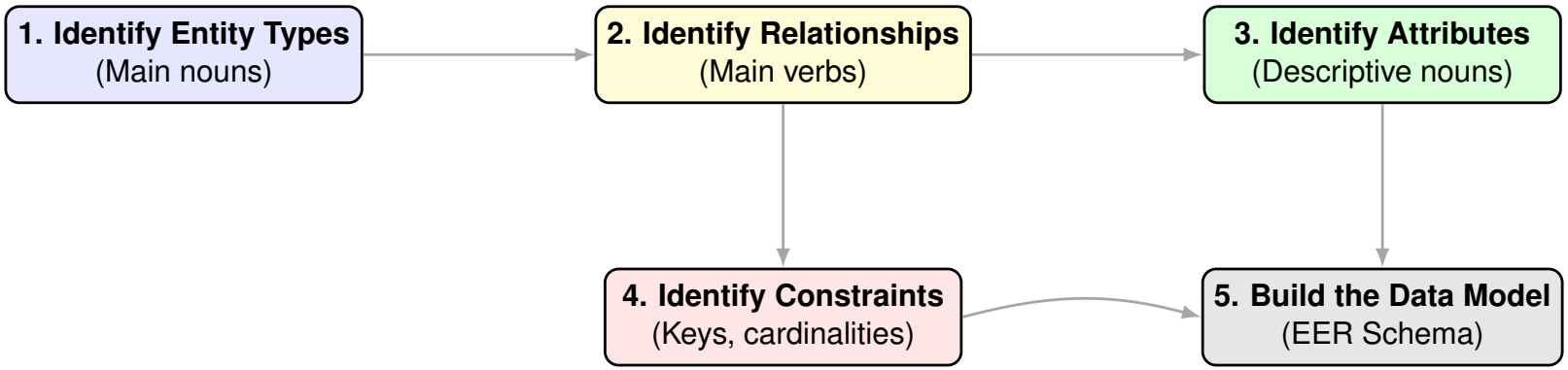
- **Entities and Classes:** Abstract representations of real-world objects or concepts.
- **Attributes:** Properties describing the characteristics of entities.
- **Relationships:** Define associations between entities.
- **Taxonomies:**
  - Organize entities into hierarchical structures.
  - Support parent-child relationships for semantic classification.
- **Integrity Constraints:**
  - Ensure consistency in knowledge representation.
  - Include constraints like type hierarchies and class disjointness.

A Simplified- Motivating Scenario

Simplified data requirements:

- A company offers goods to its customers
- Customers order goods from vendors in certain quantities
- Vendors offer goods at certain prices.
- Name and address are required from each customer and for each vendor; in addition, each customer receives a unique number and an account

Basic Methodology for Database Design – Overview



From understanding the domain to producing a structured conceptual schema.

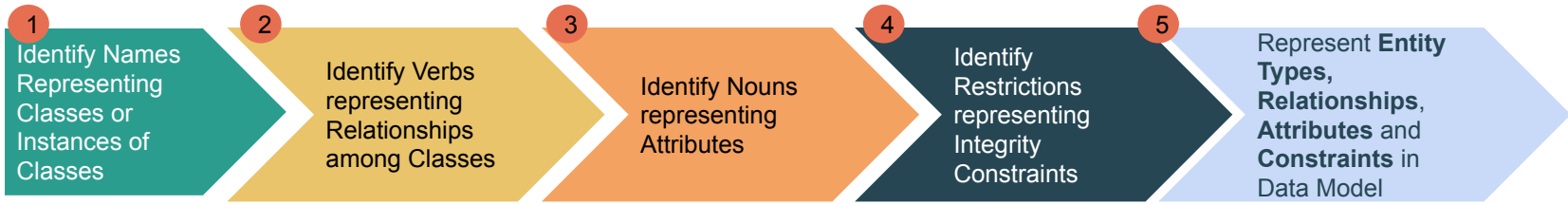
Basic Methodology for Database Design (1/3)

1. **Identify Names Representing Entity Types:**
   - Identify the main entities or objects that will be represented in the database. These entities could be real-world objects, concepts, or events for which information will be represented.
2. **Identify Verbs Representing Relationships among Entity Types:**
   - Determine how the identified entities are related to each other. Relationships are typically expressed through verbs and describe how entities interact or are associated with each other. Relationships can be one-to-one, one-to-many, or many-to-many.

Basic Methodology for Database Design (2/3)

3. **Identify Nouns Representing Attributes:**
   - Attributes are the properties or characteristics of entities. They describe the data that you want to store about each entity. These could include things like names, ages, dates, quantities, etc. Each attribute becomes a column within the respective entity's table.
4. **Identify Restrictions Representing Integrity Constraints:**
   - Integrity constraints define rules that data in the database must adhere to in order to maintain consistency and accuracy. These constraints ensure the validity and reliability of the data. Examples include primary key constraints, foreign key constraints, unique constraints, and check constraints.

Basic Methodology for Database Design (3/3)



5. **Represent Entity Types, Relationships, Attributes, and Constraints in Data Model:**
   - Elements identified in the previous steps are represented in a formal data model.

A Simplified- Motivating Scenario

Simplified data requirements:

- A company offers goods to its customers
- Customers order goods from vendors in certain quantities
- Vendors offer goods at certain prices.
- Name and address are required from each customer and for each vendor; in addition, each customer receives a unique number and an account

Entity types
Relationships
Attributes

A Simplified- Motivating Scenario

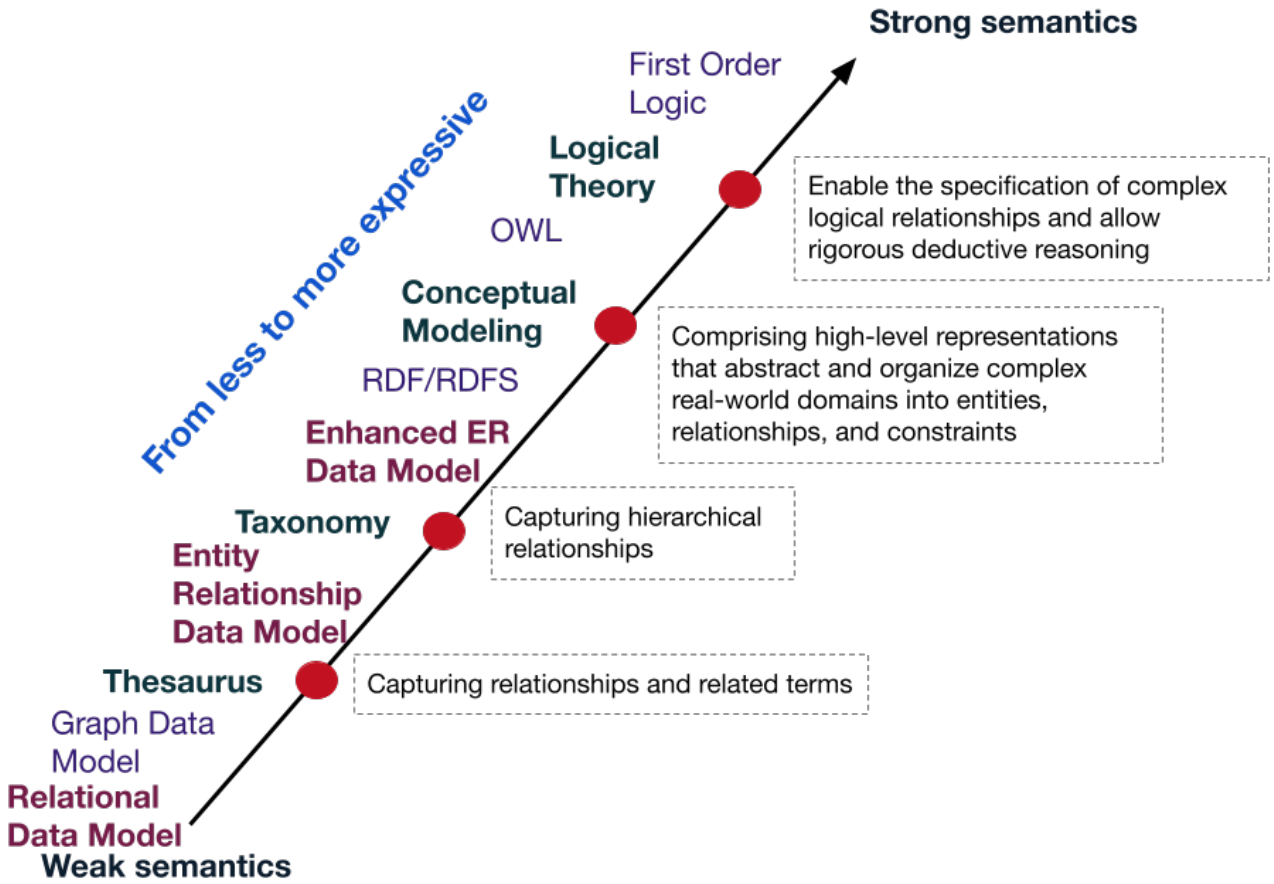
Simplified data requirements:

- A company offers goods to its customers
- Customers order goods from vendors in certain quantities
- Vendors offer goods at certain prices.
- Name and address are required from each customer and for each vendor; in addition, each customer receives a unique number and an account

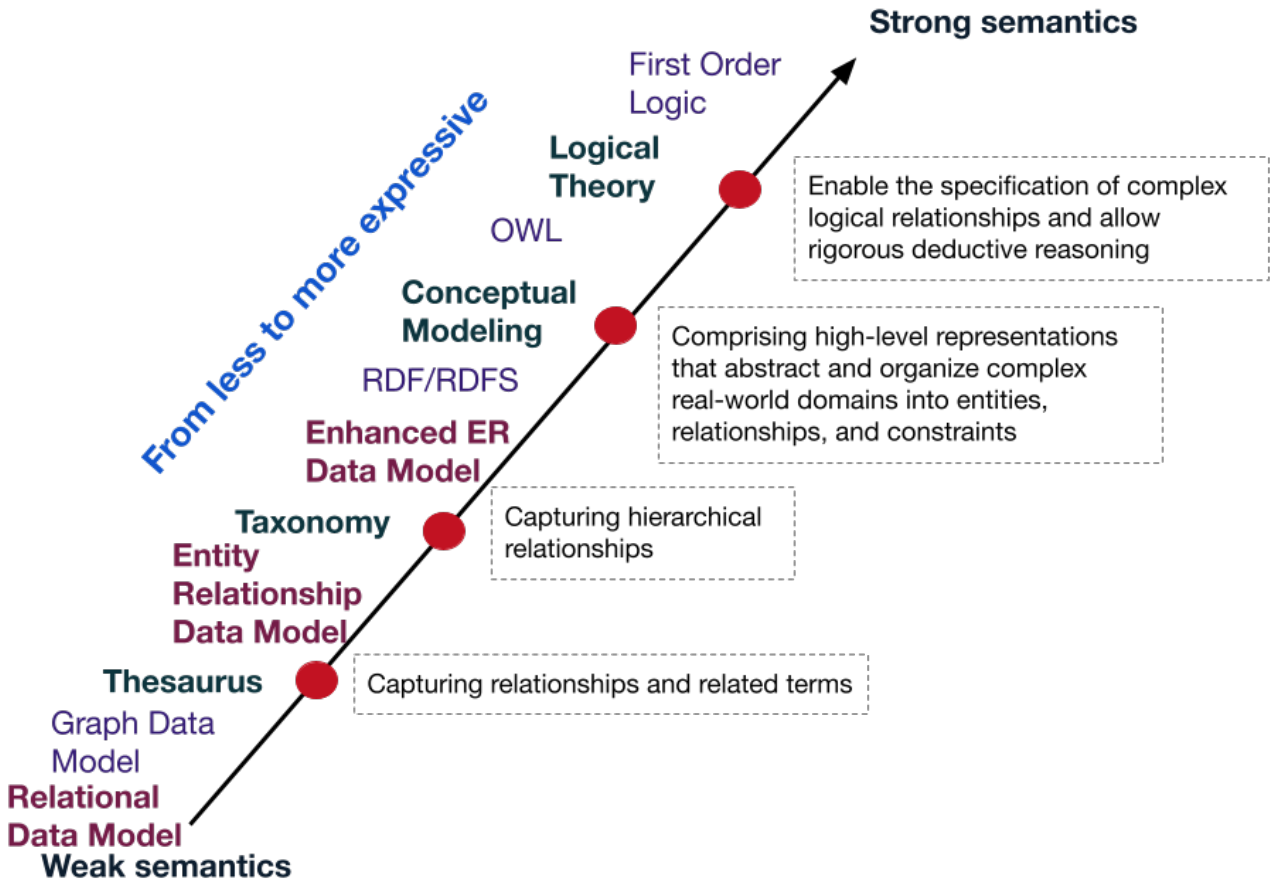
Entity types
Relationships
Attributes

How can these concepts model be modeled using a formal system?

Spectrum of Data Models (Lecture 2)

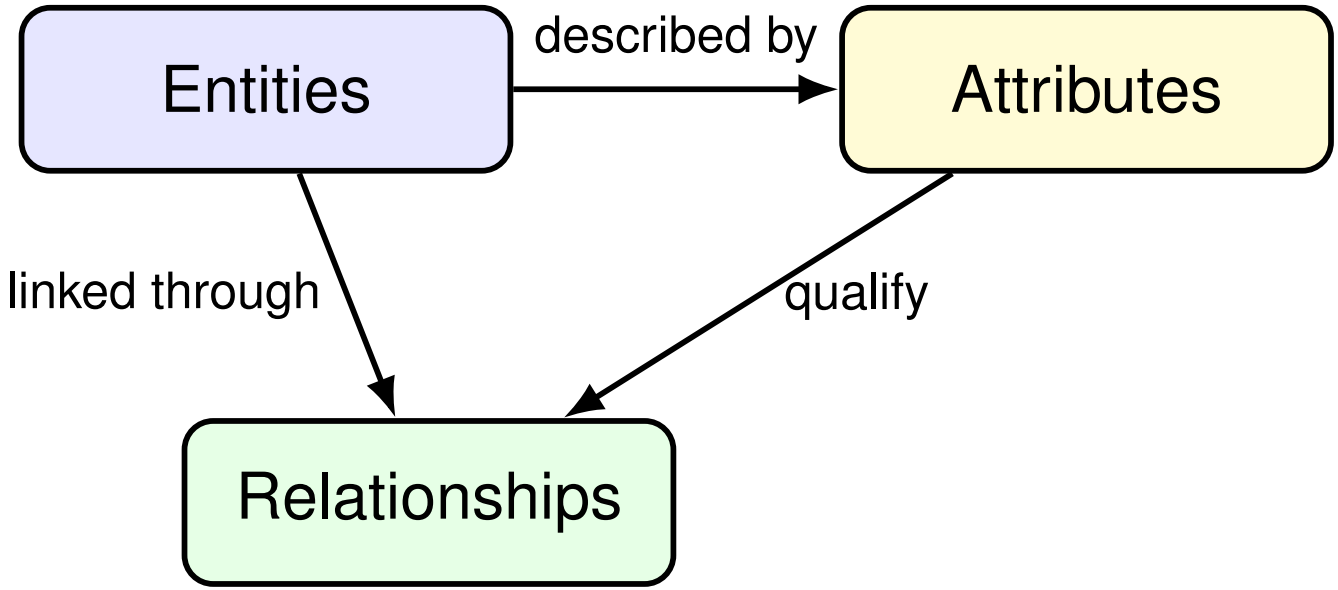


Spectrum of Data Models (Lecture 2)



Our focus on Basic Concepts modeled using EER and their representations in RDF, RDFS, and OWL.

Basic Concepts – Overview



The core of semantic models: entities, attributes, and relationships form structured meaning.

Basic Concepts for Semantic Modeling [1]

Entity Types or clases: A group of *entities* that share the same properties, e. g., STUDENT, PROFESSOR, COURSE . . .

Meaning:

> Entity type or class **E**
> $\mu(E)$ = the set of all instances (possible entities) of type E, representing the complete collection of identifiable entities under that type.

Representing Entity Types/Classes: An entity type is *E* displayed in a rectangular box.

Attributes (1/2) [1]

Attributes: properties of entities or relations between attributes

- Simple: the attribute takes a single and atomic value.
  `NameTeam` is an atomic attribute that takes single values.

Attributes (1/2) [1]

Attributes: properties of entities or relations between attributes

- Simple: the attribute takes a single and atomic value.
  `NameTeam` is an atomic attribute that takes single values.
- Composite: the attribute is composed of several components. Composition may form a hierarchy where some components are themselves composite.
  `Address` is composed of the five attributes `Street, Number, City, State, ZipCode, Country`.
  `Name` is composed of three attributes `FirstName, MiddleName, LastName`.

Attributes (1/2) [1]

Attributes: properties of entities or relations between attributes

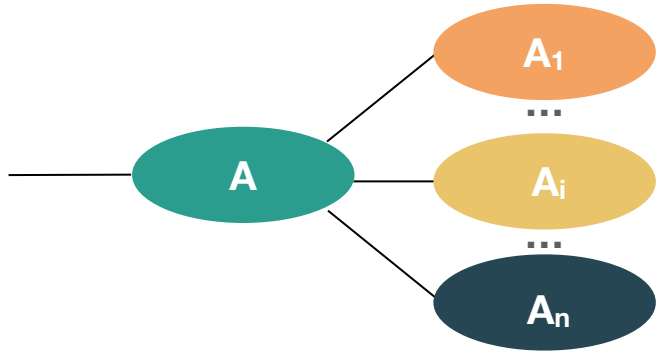
- Simple: the attribute takes a single and atomic value.
  `NameTeam` is an atomic attribute that takes single values.
- Composite: the attribute is composed of several components. Composition may form a hierarchy where some components are themselves composite.
  `Address` is composed of the five attributes `Street, Number, City, State, ZipCode, Country`.
  `Name` is composed of three attributes `FirstName, MiddleName, LastName`.
- Multi-valued: the attribute may have multiple values.
  `PhoneNumber` of a person.

Attributes (1/2) [1]

Attributes: properties of entities or relations between attributes

- Simple: the attribute takes a single and atomic value.
  `NameTeam` is an atomic attribute that takes single values.
- Composite: the attribute is composed of several components. Composition may form a hierarchy where some components are themselves composite.
  `Address` is composed of the five attributes `Street, Number, City, State, ZipCode, Country`.
  `Name` is composed of three attributes `FirstName, MiddleName, LastName`.
- Multi-valued: the attribute may have multiple values.
  `PhoneNumber` of a person.
- Optional: the attribute may take no values.
  `PersonDegree` of a person.

Attributes (2/2) [1]

Type of Attribute	Notation
Simple	
Composite	
Multi-valued	
Optional	

Same notation is followed to represent attributes in relationships and classes.

Attributes- Semantics

Let A be an attribute of the entity/relationship type E with domains $D_1, \ldots, D_k$. A is a function that assigns to each element in $\mu(E)$ values in $\mathcal{P}(\mathcal{P}(D_1) \times \cdots \times \mathcal{P}(D_k))$.

Attributes- Semantics

Let A be an attribute of the entity/relationship type E with domains $D_1, \ldots, D_k$. A is a function that assigns to each element in $\mu(E)$ values in $\mathcal{P}(\mathcal{P}(D_1) \times \cdots \times \mathcal{P}(D_k))$.

- Simple: the attribute takes a single and atomic value. k is equal to 1 and the function A only goes to singleton sets of elements in D_1.

Attributes- Semantics

Let A be an attribute of the entity/relationship type E with domains $D_1, \ldots, D_k$. A is a function that assigns to each element in $\mu(E)$ values in $\mathcal{P}(\mathcal{P}(D_1) \times \cdots \times \mathcal{P}(D_k))$.

- Simple: the attribute takes a single and atomic value. k is equal to 1 and the function A only goes to singleton sets of elements in D_1.
- Composite: A is composed of attributes, thus, k is greater than 1.

Attributes- Semantics

Let A be an attribute of the entity/relationship type E with domains $D_1, \ldots, D_k$. A is a function that assigns to each element in $\mu(E)$ values in $\mathcal{P}(\mathcal{P}(D_1) \times \cdots \times \mathcal{P}(D_k))$.

- Simple: the attribute takes a single and atomic value. k is equal to 1 and the function A only goes to singleton sets of elements in D_1.
- Composite: A is composed of attributes, thus, k is greater than 1.
- Multi-valued: the attribute may have multiple values. The function can take values in subsets of $\mathcal{P}(\mathcal{P}(D_1) \times \cdots \times \mathcal{P}(D_k))$ of cardinality greater than 1.

Attributes- Semantics

Let A be an attribute of the entity/relationship type E with domains $D_1, \ldots, D_k$. A is a function that assigns to each element in $\mu(E)$ values in $\mathcal{P}(\mathcal{P}(D_1) \times \cdots \times \mathcal{P}(D_k))$.

- Simple: the attribute takes a single and atomic value. k is equal to 1 and the function A only goes to singleton sets of elements in D_1.
- Composite: A is composed of attributes, thus, k is greater than 1.
- Multi-valued: the attribute may have multiple values. The function can take values in subsets of $\mathcal{P}(\mathcal{P}(D_1) \times \cdots \times \mathcal{P}(D_k))$ of cardinality greater than 1.
- Optional: the attribute may take no values. Each domain D_i includes the *null set*.

Attributes- Semantics

Let A be an attribute of the entity/relationship type E with domains $D_1, \ldots, D_k$. A is a function that assigns to each element in $\mu(E)$ values in $\mathcal{P}(\mathcal{P}(D_1) \times \cdots \times \mathcal{P}(D_k))$.

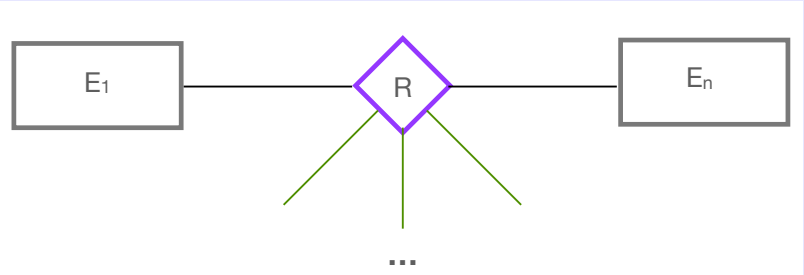
- Simple: the attribute takes a single and atomic value. k is equal to 1 and the function A only goes to singleton sets of elements in D_1.
- Composite: A is composed of attributes, thus, k is greater than 1.
- Multi-valued: the attribute may have multiple values. The function can take values in subsets of $\mathcal{P}(\mathcal{P}(D_1) \times \cdots \times \mathcal{P}(D_k))$ of cardinality greater than 1.
- Optional: the attribute may take no values. Each domain D_i includes the *null set*.

An attribute <u>A</u> represents that A is a key, i.e., meaning it uniquely identifies each entity that has this attribute.

Relationships [1]

Relationship Types:

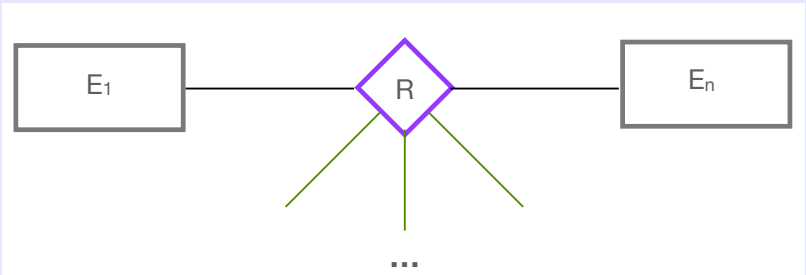
A diamond-shaped box displays a relationship, connecting to the participating entity types via straight lines from the diamond to the rectangular boxes.



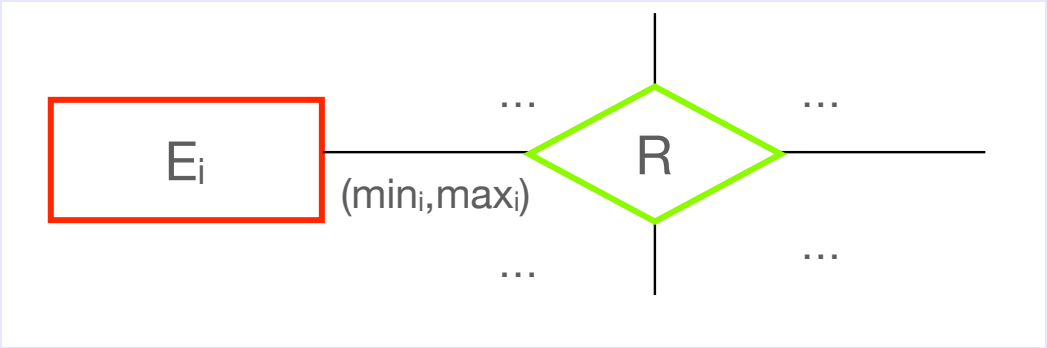
Relationships [1]

Relationship Types:

A diamond-shaped box displays a relationship, connecting to the participating entity types via straight lines from the diamond to the rectangular boxes.



Cardinality of a Relationship:



For all $e_i \in \mu(E_i)$, each entity from E_i is involved in min_i to max_i instances of $\mu(R)$:

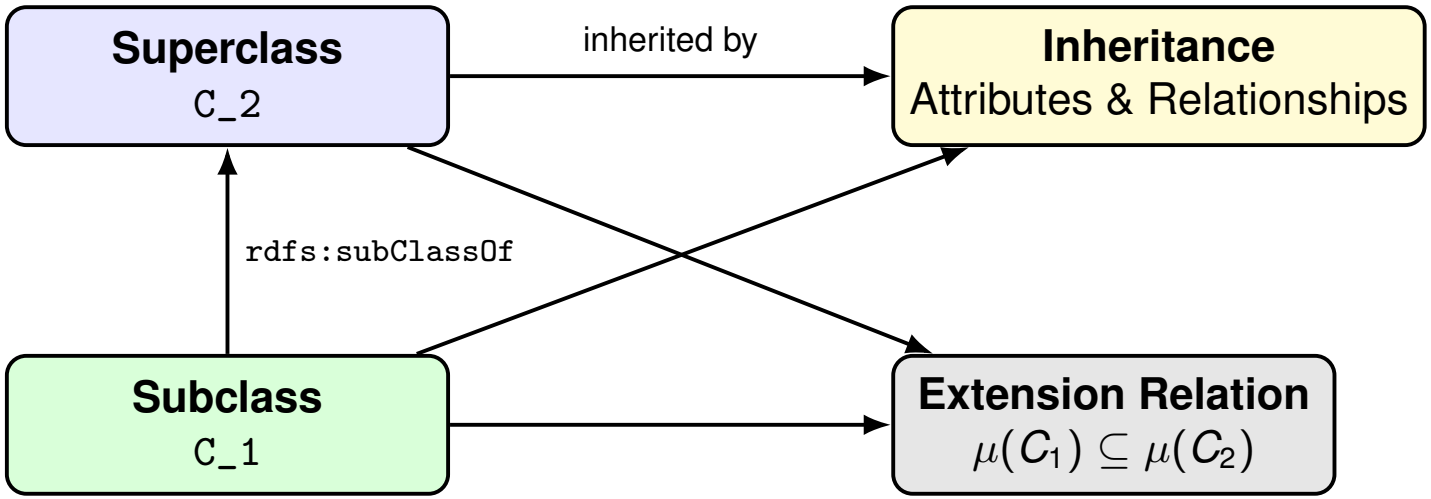
$$min_i \leq |\{r \in \mu(R) : r = (\ldots, e_i, \ldots)\}| \leq max_i$$

default: (0, *), i.e., no restriction

Taxonomies – Overview

Classes, Subclasses, and Inheritance

A taxonomy organizes classes into a hierarchy where **subclasses inherit** the attributes and relationships of their **superclasses**. Extensions follow the rule: $\mu(C_1) \subseteq \mu(C_2)$.



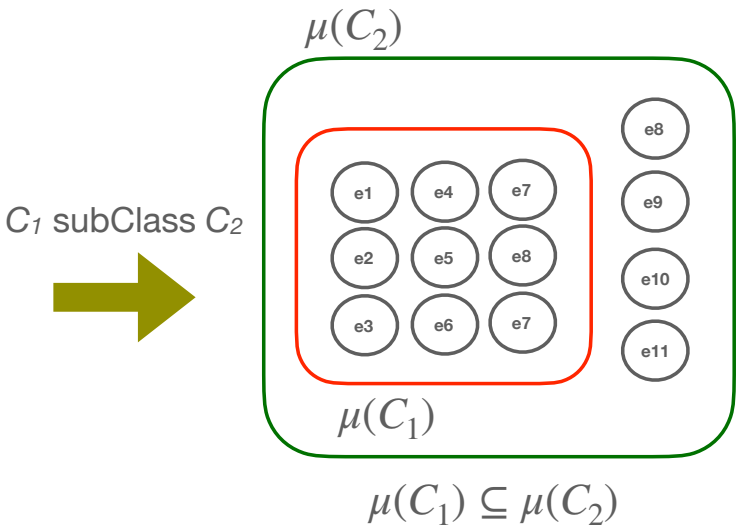
Subclassing = structural inheritance + semantic inclusion of instances.

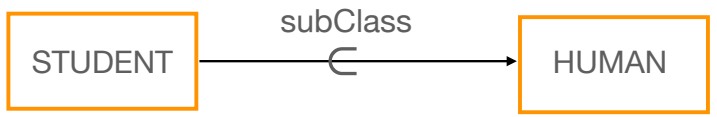
Taxonomies (1/3) [1]

- Class C instead of Entity type C
- If C_1 is subclass of C_2:
  - C_1 and C_2 are classes
  - C_2 is called the superclass of C_1
  - C_1 inherits all the attributes and relationship of the superclass C_2
  - If $\mu(C_1)$ and $\mu(C_2)$ are extensions of C_1 and C_2, respectively:
    - $\mu(C_1)$ is subset of $\mu(C_2)$, i.e., $\mu(C_1) \subseteq \mu(C_2)$

Taxonomies (1/3) [1]

- Class C instead of Entity type C
- If C_1 is subclass of C_2:
  - C_1 and C_2 are classes
  - C_2 is called the superclass of C_1
  - C_1 inherits all the attributes and relationship of the superclass C_2
  - If $\mu(C_1)$ and $\mu(C_2)$ are extensions of C_1 and C_2, respectively:
    - $\mu(C_1)$ is subset of $\mu(C_2)$, i.e., $\mu(C_1) \subseteq \mu(C_2)$





STUDENT *subClass* HUMAN

Taxonomies (2/3) [1]

Given a set of classes $\{C_1, C_2, \ldots, C_n\}$ with the same superclass C

- C is called a generalization of $C_1, C_2, \ldots, C_n$.
- $C_1, C_2, \ldots, C_n$ are the specialization of C.

Taxonomies (2/3) [1]

Given a set of classes $\{C_1, C_2, \ldots, C_n\}$ with the same superclass C

- C is called a generalization of $C_1, C_2, \ldots, C_n$.
- $C_1, C_2, \ldots, C_n$ are the specialization of C.

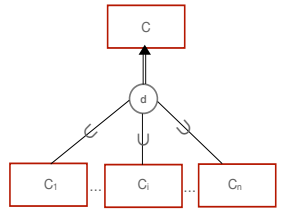
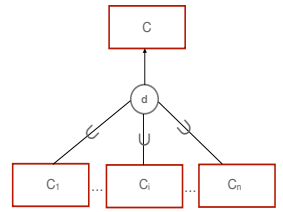
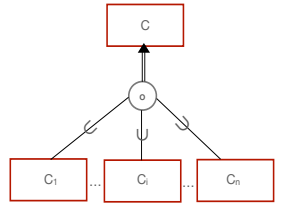
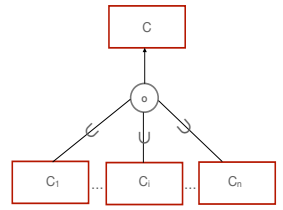
Completeness

- Total:
  $C = C_1 \cup C_2 \cup \ldots \cup C_n$
- Partial:
  $C_1 \cup C_2 \cup \ldots \cup C_n \subset C$

Disjointness

- Disjoint:
  $\forall\, i, j$, such as $i \neq j$, $Ci \cap Cj = \emptyset$
- Overlapping:
  Not for all i, j, such as $i \neq j$, $Ci \cap Cj = \emptyset$

Taxonomies- EER Notation (3/3) [1]

Specialization Type	Notation
Disjoint and Total Participation	
Disjoint and Partial Participation	
Overlap and Total Participation	
Overlap and Partial Participation	

Example 1: Universe of Discourse (1/7)

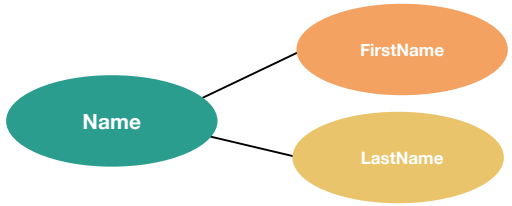
- Students and lecturers are persons. A person has a unique identifier, a name compose of first and last name, and may have several email addresses.
- Students are either undergraduate or graduate, but not both.
- Lecturers can exclusively be professors, instructors, or hiwis. Hiwis are students.
- Students are identified by a matriculation number.
- Professors supervise hiwis and instructors.
- A course has unique identifier and a title.
- Semesters have a unique identifier and a title.
- Students are enrolled in courses in given semester. Similarly, courses are taught by lecturers in a given semester.

Example 1: Universe of Discourse (2/7)

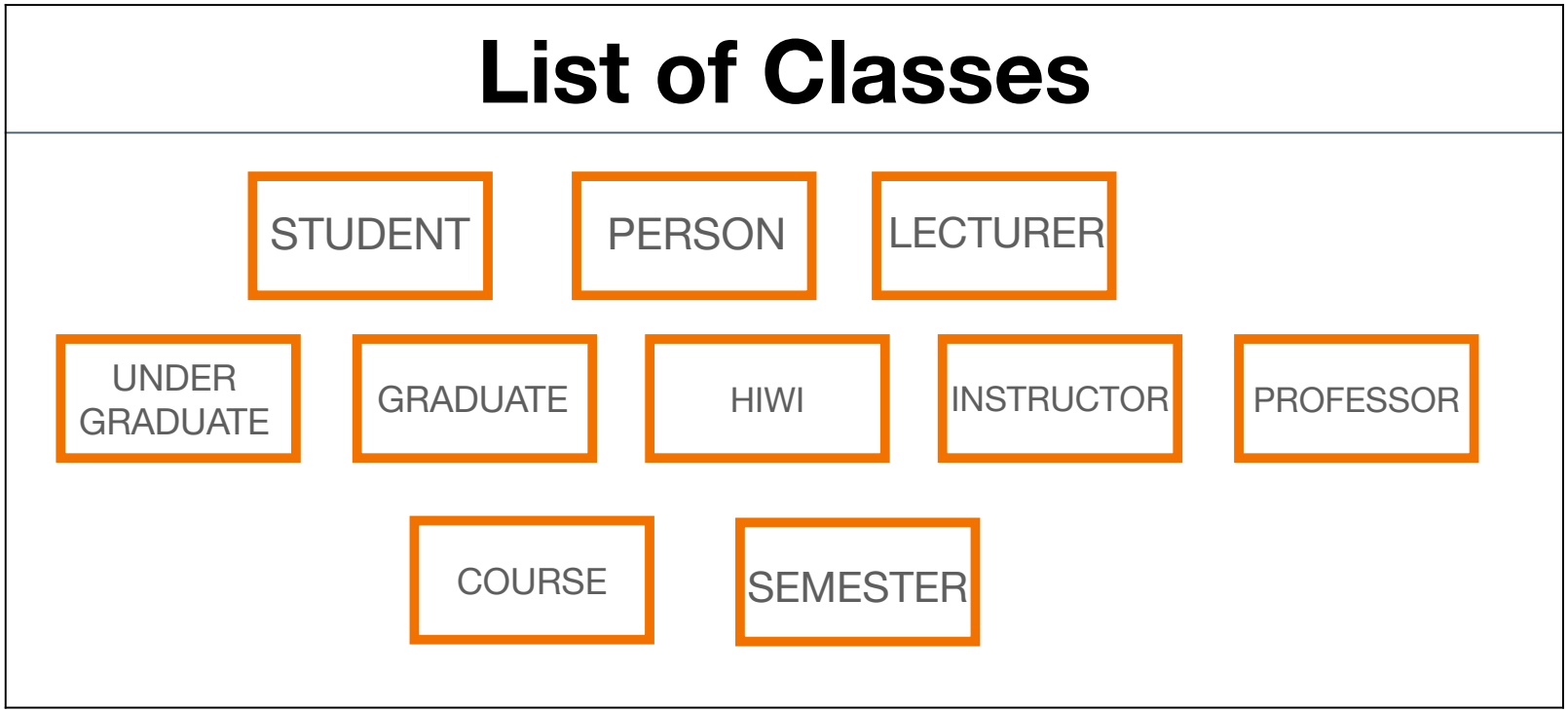
- Students and lecturers are persons. A person has a unique identifier, a name compose of first and last name, and may have several email addresses.
- Students are either undergraduate or graduate, but not both.
- Lecturers can exclusively be professors, instructors, or hiwis. Hiwis are students.
- Students are identified by a matriculation number.
- Professors supervise hiwis and instructors.
- A course has unique identifier and a title.
- Semesters have a unique identifier and a title.
- Students are enrolled in courses in given semester. Similarly, courses are taught by lecturers in a given semester.

Yellow words represent instances of the IS-A relationship.

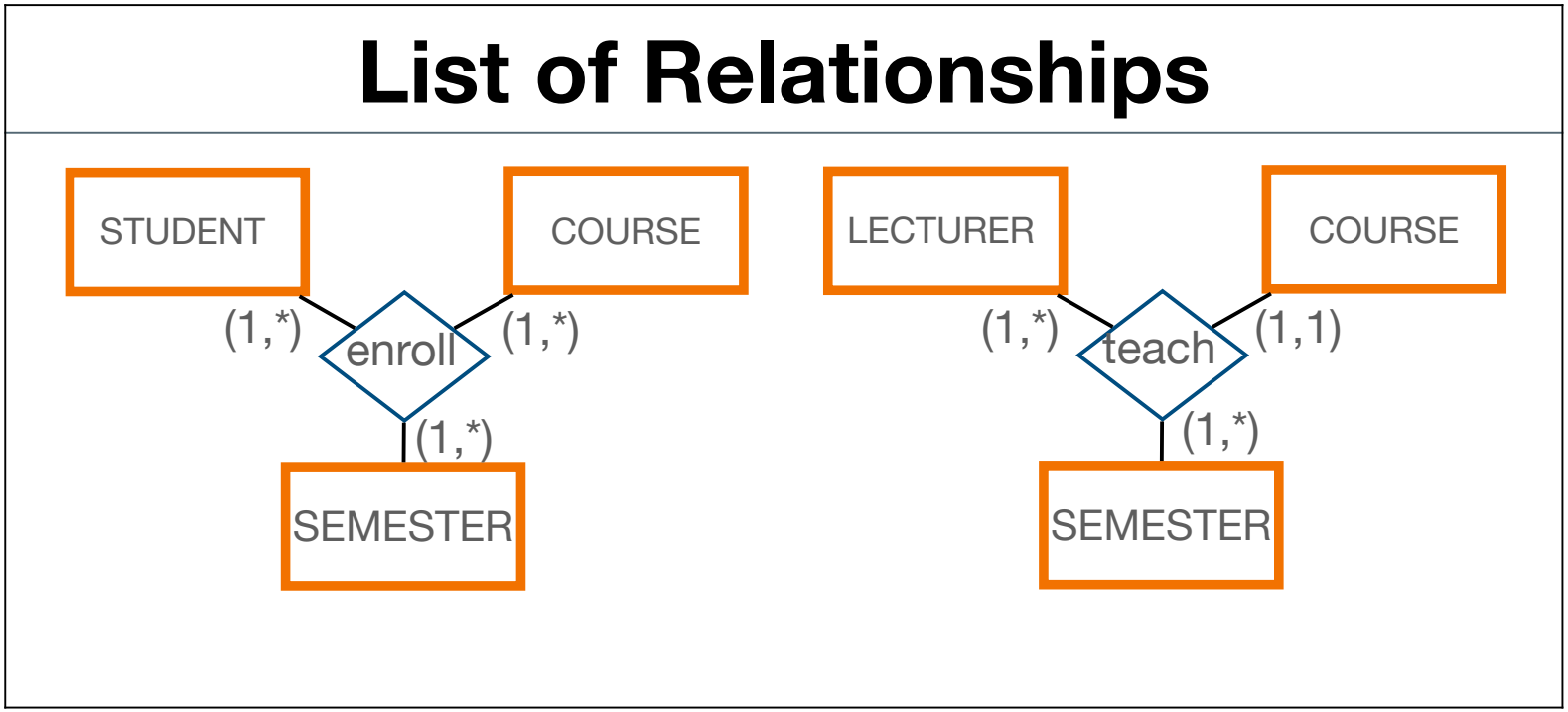
Example 1: Attributes (3/7)

List of Attributes	Representation
Simple	
Composite	
Multi-valued	

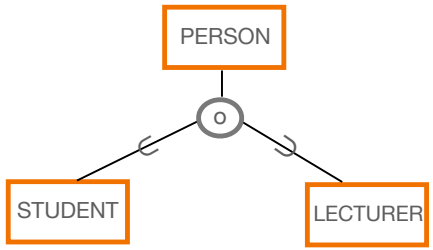
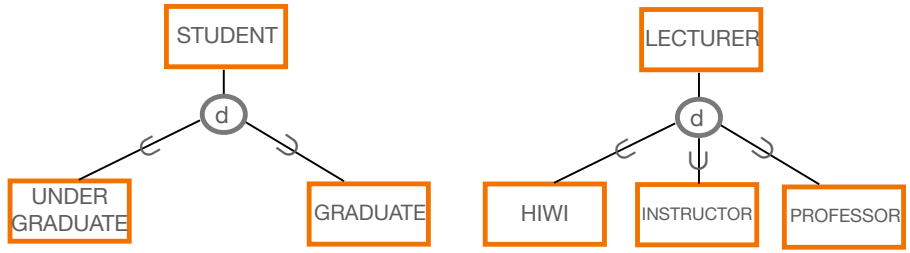
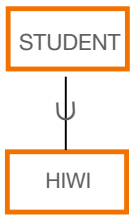
Example 1: Classes (4/7)



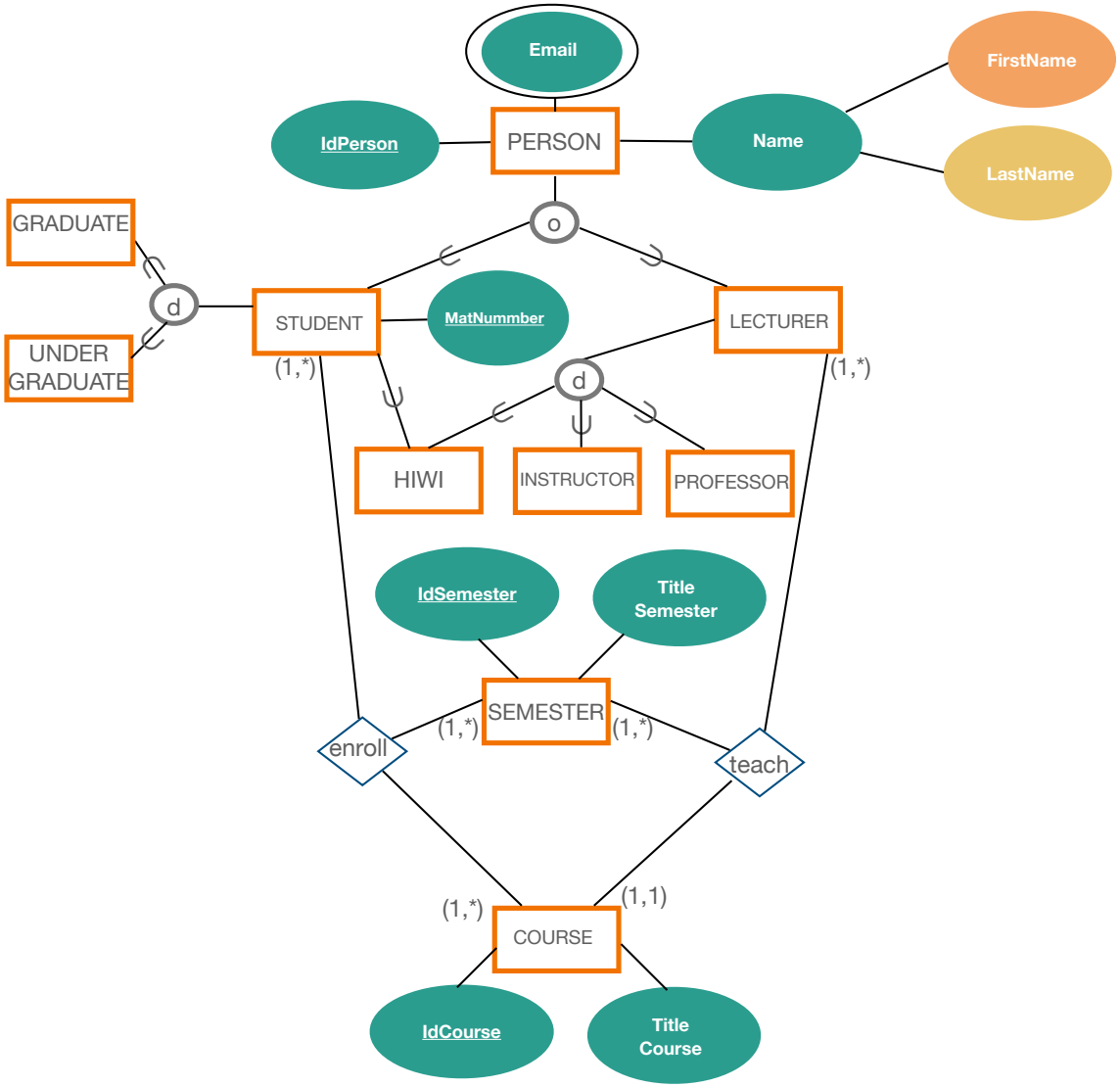
Example 1: Relationships (5/7)



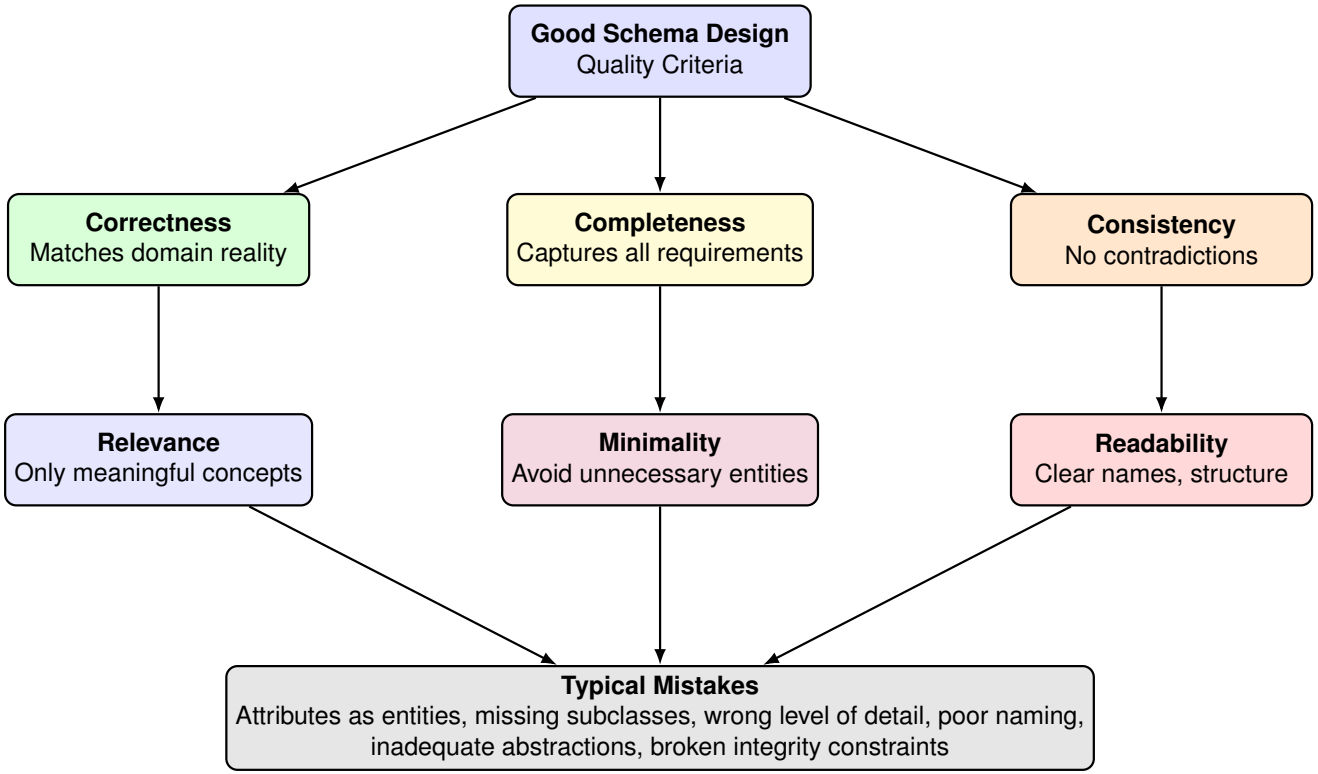
Example 1: Taxonomies (6/7)

Type of Taxonomies	Notation
Overlap and Partial Participation	
Disjoint and Partial Participation	
SubClassOf	

Example 1: Complete Schema (7/7)



Quality Criteria & Typical Mistakes – Overview



Quality criteria guide good design; mistakes violate them.

Quality Criteria for Good Designs (1/3)

- Correctness

  Are all modeled concepts correct with respect to the requirements and conditions of the domain?
- Completeness

  Is the schema complete with respect to the data requirements?
- Consistency

  Are all modelled concepts free of contradictions?
- Relevance

  Are technically relevant all the concepts represented in the schema?

Quality Criteria for Good Designs (2/3)

- Scope

  Is the scope of the schema appropriate with respect to the represented mini-world?
- Extensibility/Adaptability

  Is the schema easily extensible for future needs?
- Level of Detail

  Is the level of detail of the modelled concepts appropiate?
- Minimality

  Are the modeled concepts adequately represented by the data model structures?
- Readability

  Are all explained terms used in the schema?

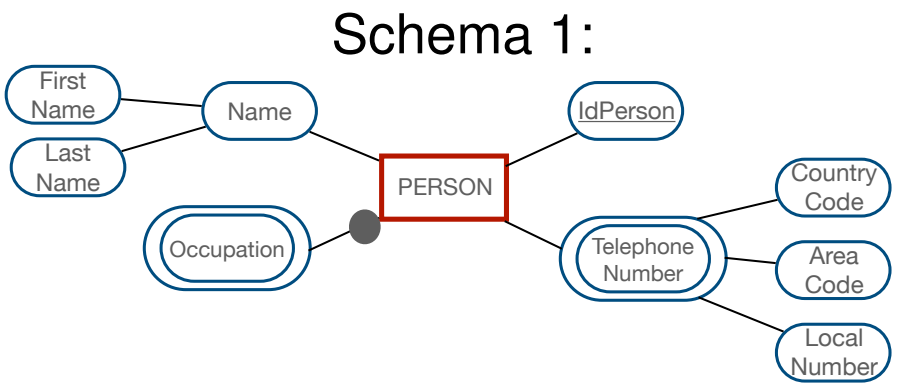
Quality Criteria for Good Designs (3/3)

- Proper Naming

  Names should be:
  - Self-documented
  - With clear relation with the concepts in the universe of discourse
  - Should not be too long
- Entity-Types

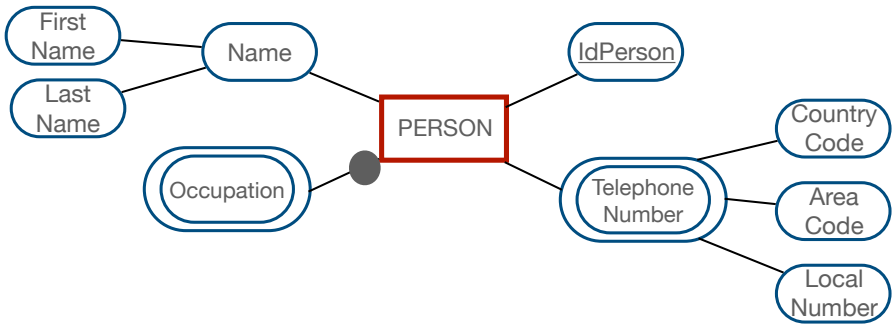
  Nouns in singular form
- Relationships
  - Should correspond to verbs
  - By default relationship names should be read from top to bottom and from left to right.

Attributes versus Entity Types (1/3)

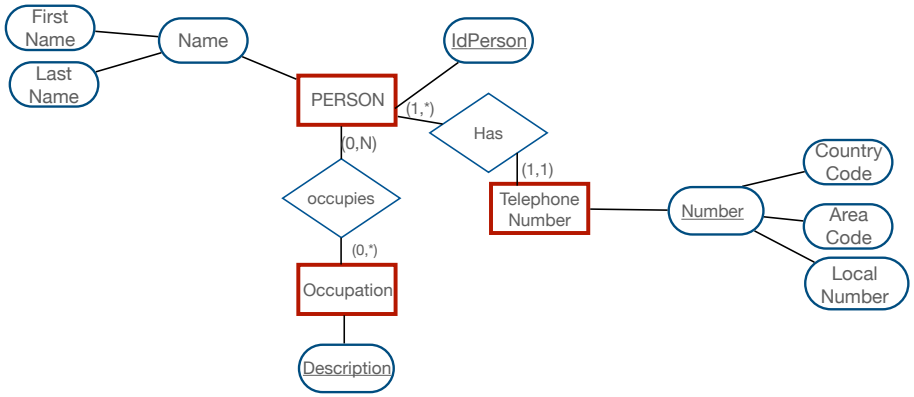


Attributes versus Entity Types (1/3)

Schema 1:

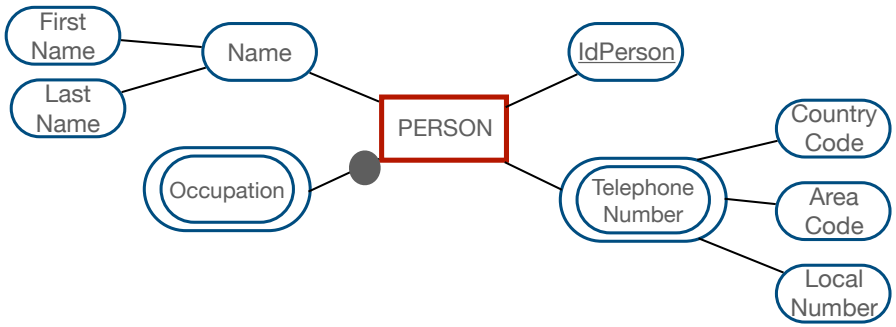


Schema 2:

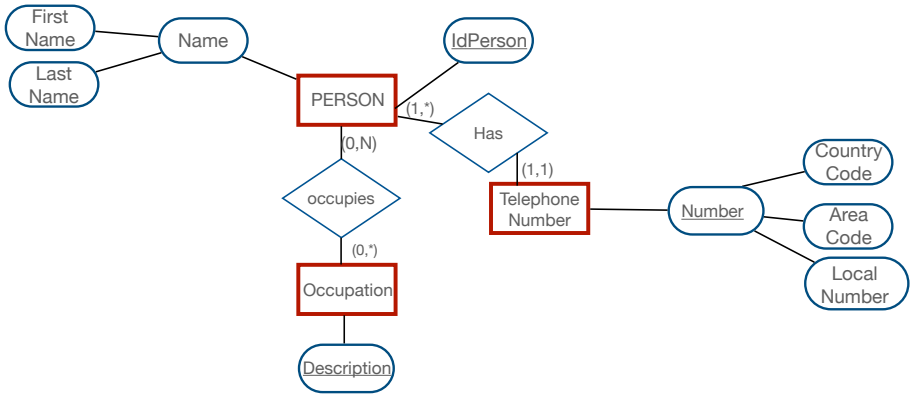


Attributes versus Entity Types (1/3)

Schema 1:



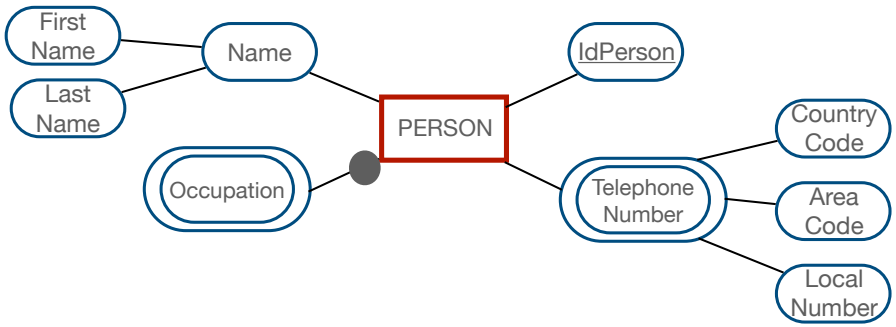
Schema 2:



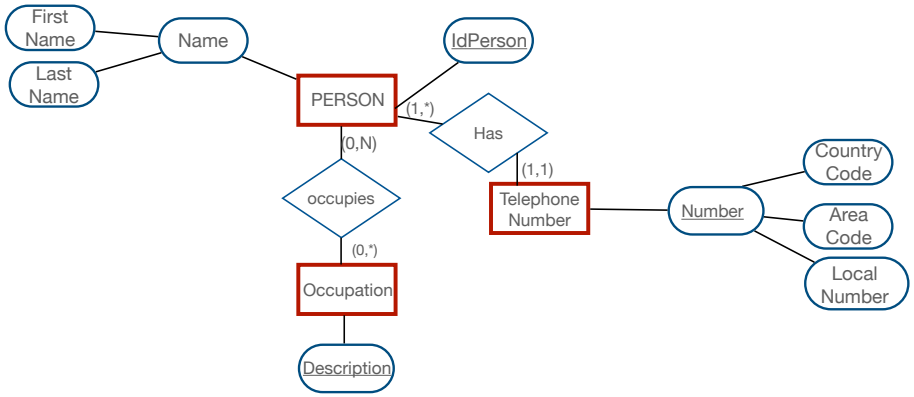
Schema 1 represents that occupation and telephone number are features of entity type PERSON. This schema respects quality criteria better than Schema 2.

Attributes versus Entity Types (1/3)

Schema 1:



Schema 2:



Schema 1 represents that occupation and telephone number are features of entity type PERSON. This schema respects quality criteria better than Schema 2.

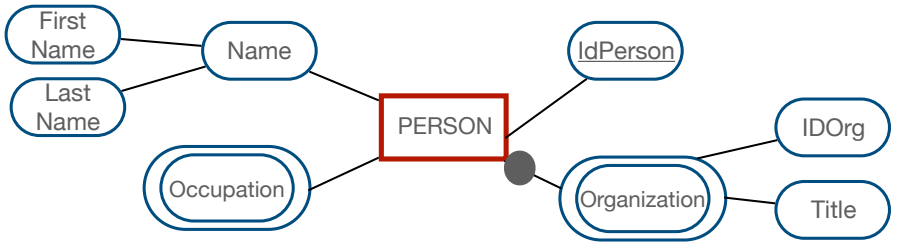
Recommendation:

Avoid entity types which are only data type values, e.g., Occupation or TelephoneNumber.

Use as entity types persons, animals, or things that play a key role in the universe of discourse.

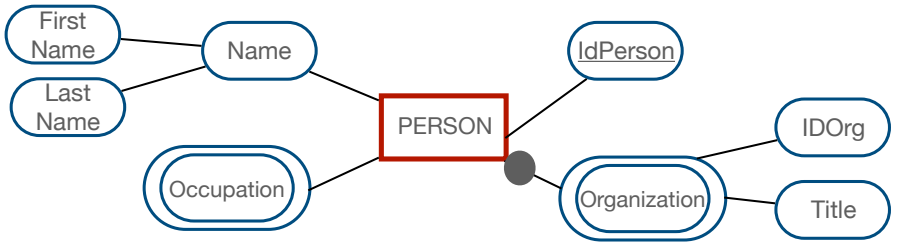
Attributes versus Entity Types (2/3)

Schema 3:

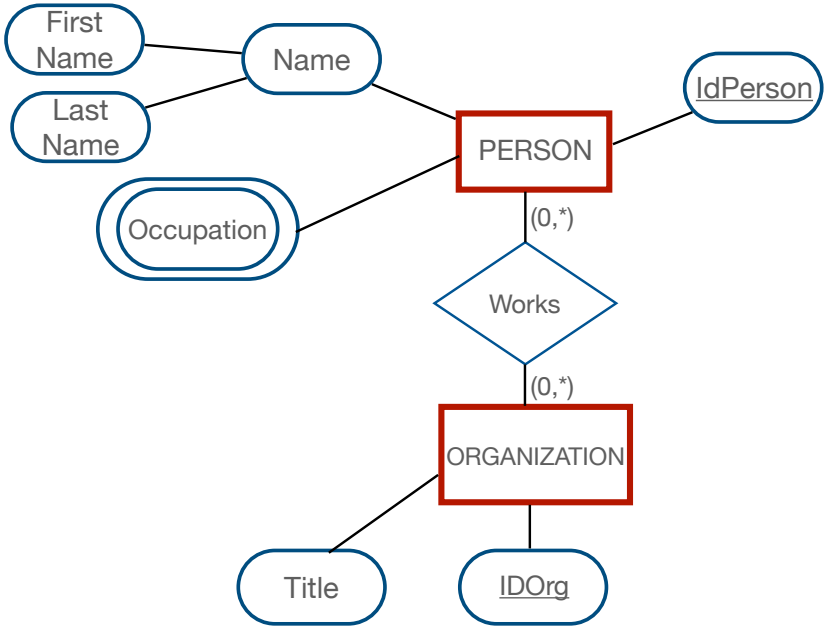


Attributes versus Entity Types (2/3)

Schema 3:

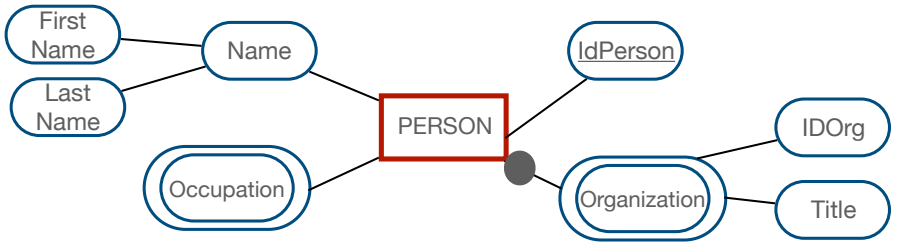


Schema 4:

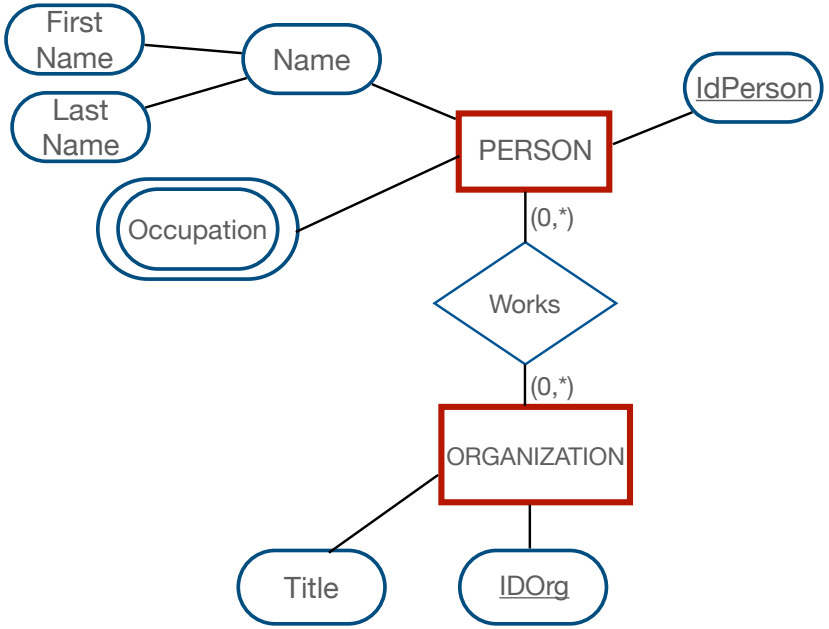


Attributes versus Entity Types (2/3)

Schema 3:



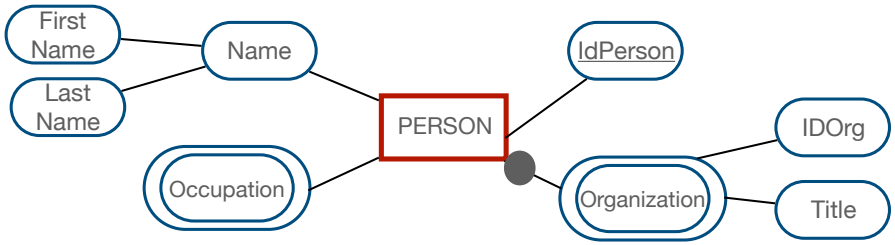
Schema 4:



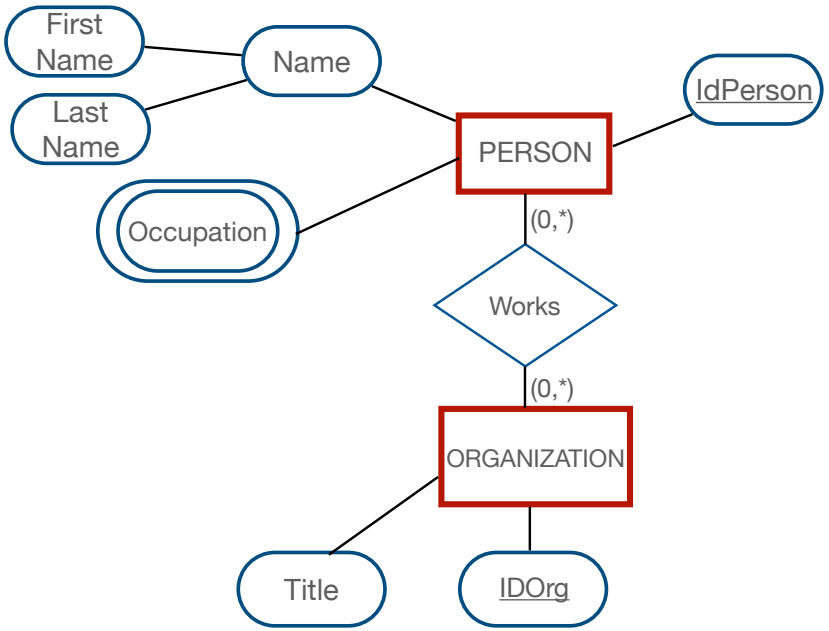
Schema 3 represents main protagonists (i.e., organizations) as attributes of other main protagonists, while Schema 4 explicitly models relevant entity types.

Attributes versus Entity Types (2/3)

Schema 3:



Schema 4:



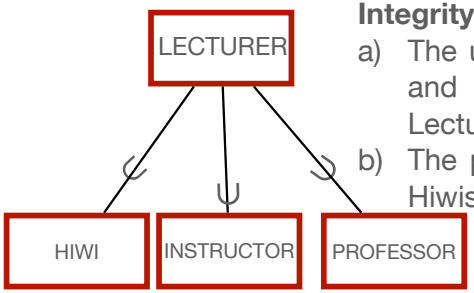
Schema 3 represents main protagonists (i.e., organizations) as attributes of other main protagonists, while Schema 4 explicitly models relevant entity types.

Recommendation:

Use as entity types persons, animals, or things that play a key role in the universe of discourse.

SubClasses versus Specialization (3/3)

Schema 5:



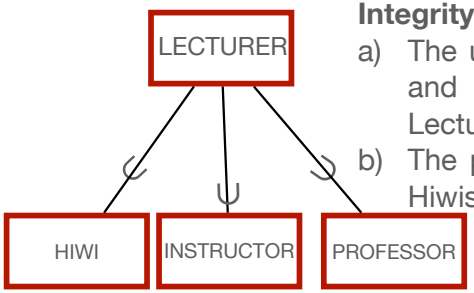
Integrity Constraints:

a) The union of the instances of Hiwis, Instructors, and Professors is equal to the instance of Lectures

b) The pair-wise intersection of the instances of Hiwis, Instructors, and Professors is empty

SubClasses versus Specialization (3/3)

Schema 5:

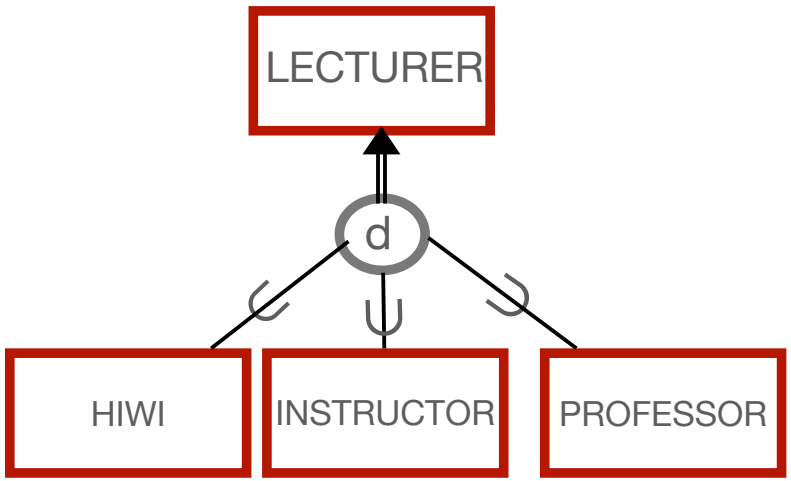


Integrity Constraints:

a) The union of the instances of Hiwis, Instructors, and Professors is equal to the instance of Lectures

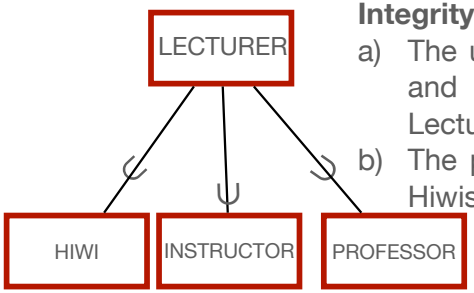
b) The pair-wise intersection of the instances of Hiwis, Instructors, and Professors is empty

Schema 6:



SubClasses versus Specialization (3/3)

Schema 5:



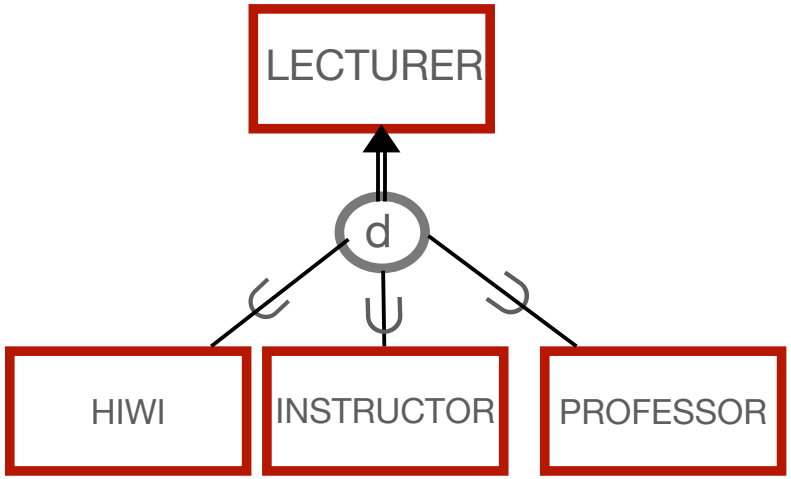
Integrity Constraints:

a) The union of the instances of Hiwis, Instructors, and Professors is equal to the instance of Lectures

b) The pair-wise intersection of the instances of Hiwis, Instructors, and Professors is empty

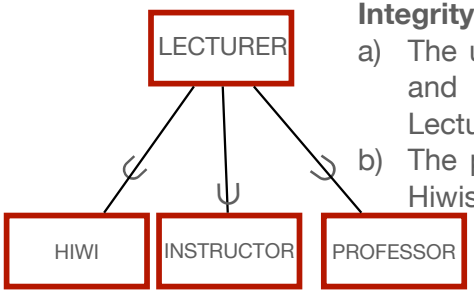
Schema 5 is not minimal because it does not adequately use the data abstractions of the EER data model

Schema 6:



SubClasses versus Specialization (3/3)

Schema 5:

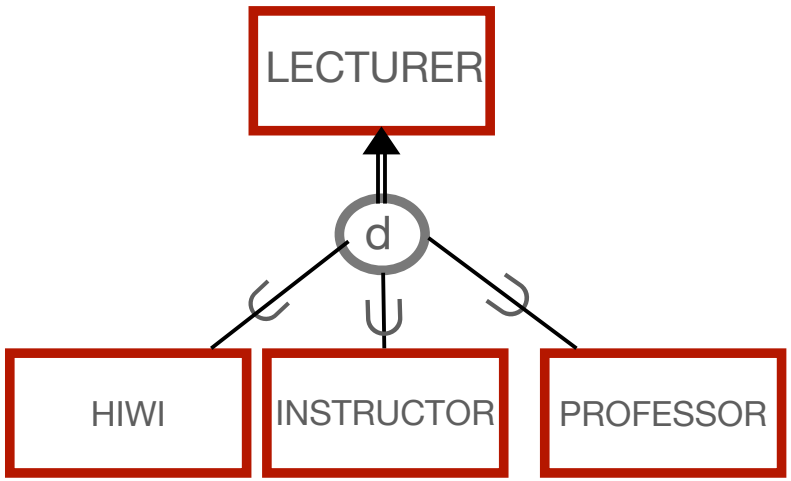


Integrity Constraints:

a) The union of the instances of Hiwis, Instructors, and Professors is equal to the instance of Lectures

b) The pair-wise intersection of the instances of Hiwis, Instructors, and Professors is empty

Schema 6:

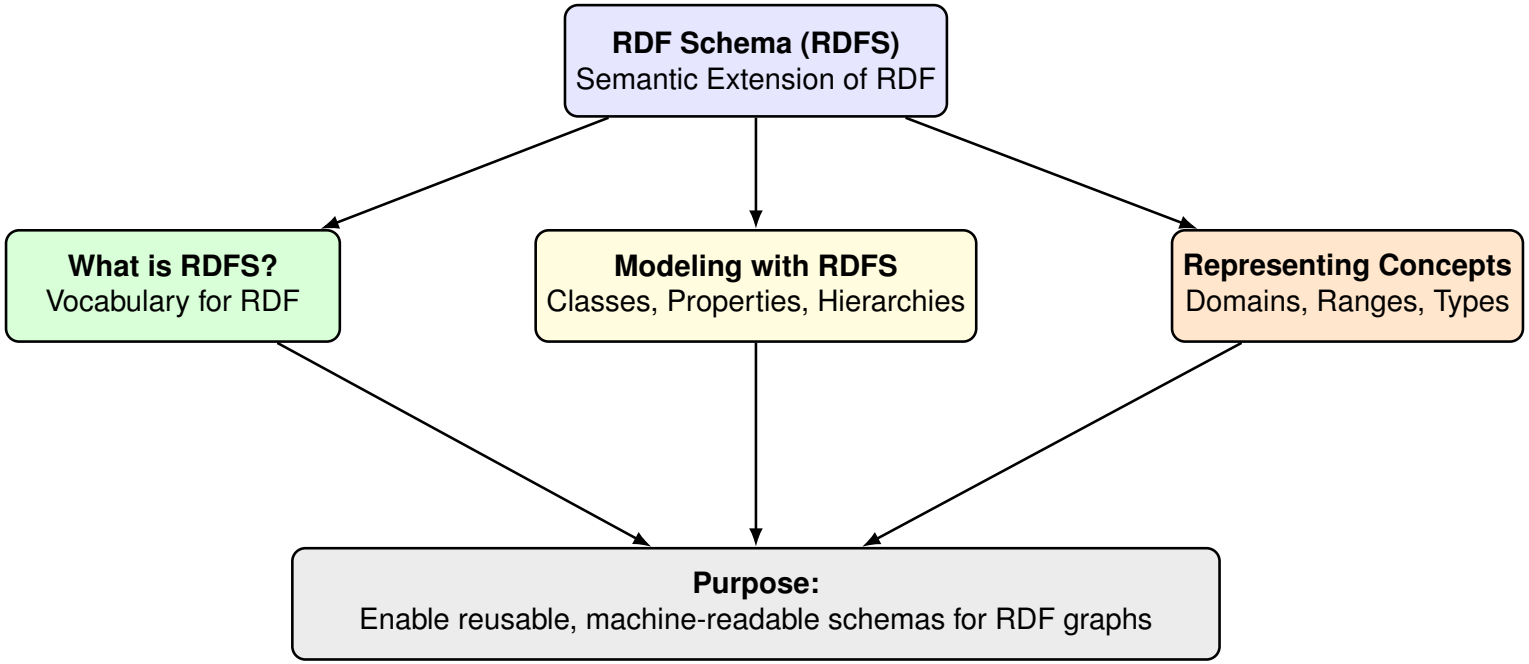


Schema 5 is not minimal because it does not adequately use the data abstractions of the EER data model

Recommendation:

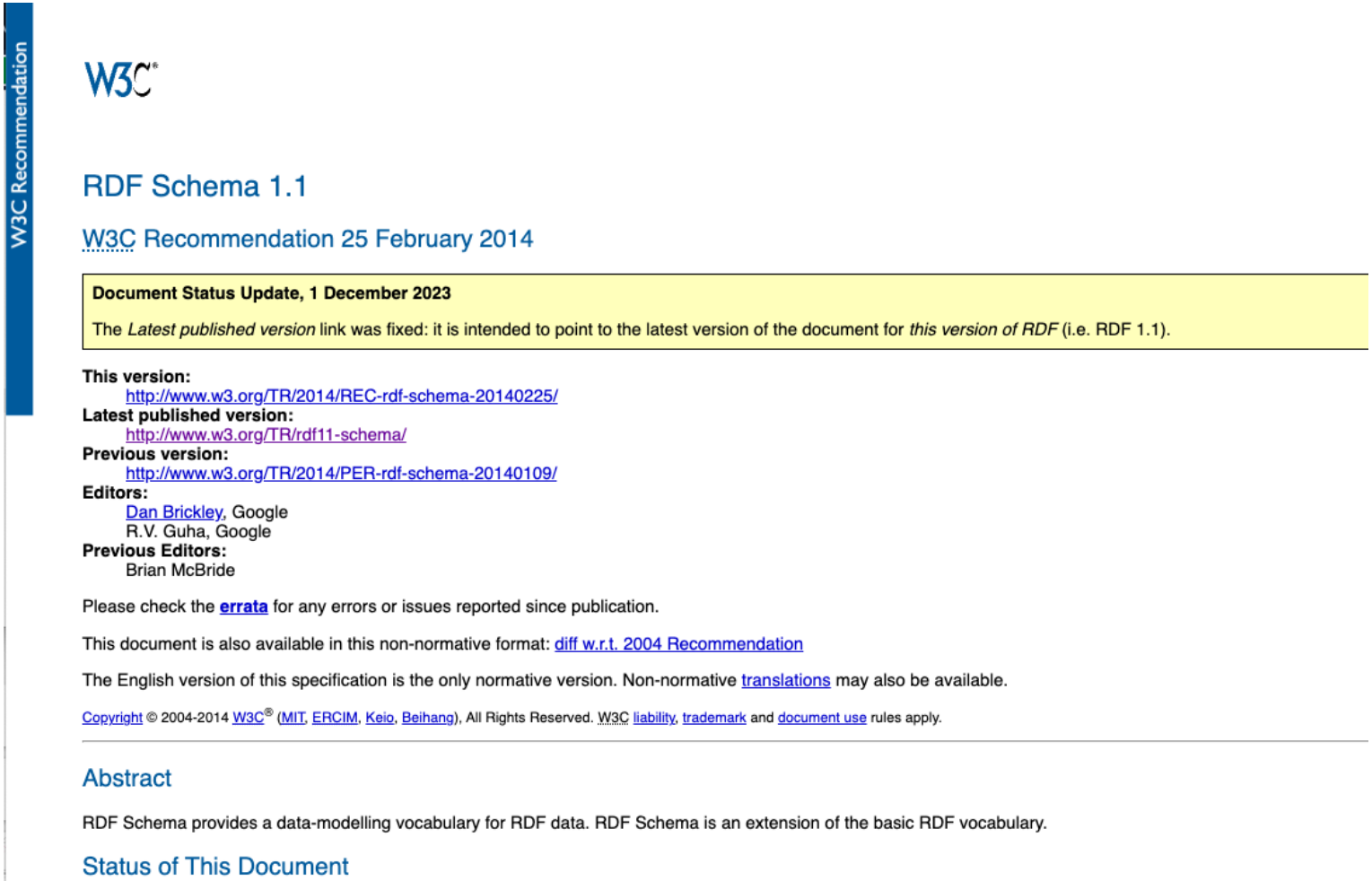
Exploit the data abstractions of the data model before adding integrity constraints.

RDF Schema (RDFS)



RDFS provides the schema layer enabling structured, interoperable RDF data.

RDFS as a W3C Standard



W3C Recommendation

W3C

RDF Schema 1.1

W3C Recommendation 25 February 2014

Document Status Update, 1 December 2023

The *Latest published version* link was fixed: it is intended to point to the latest version of the document for *this version of RDF* (i.e. RDF 1.1).

This version:
http://www.w3.org/TR/2014/REC-rdf-schema-20140225/
Latest published version:
http://www.w3.org/TR/rdf11-schema/
Previous version:
http://www.w3.org/TR/2014/PER-rdf-schema-20140109/
Editors:
Dan Brickley, Google
R.V. Guha, Google
Previous Editors:
Brian McBride

Please check the **errata** for any errors or issues reported since publication.

This document is also available in this non-normative format: diff w.r.t. 2004 Recommendation

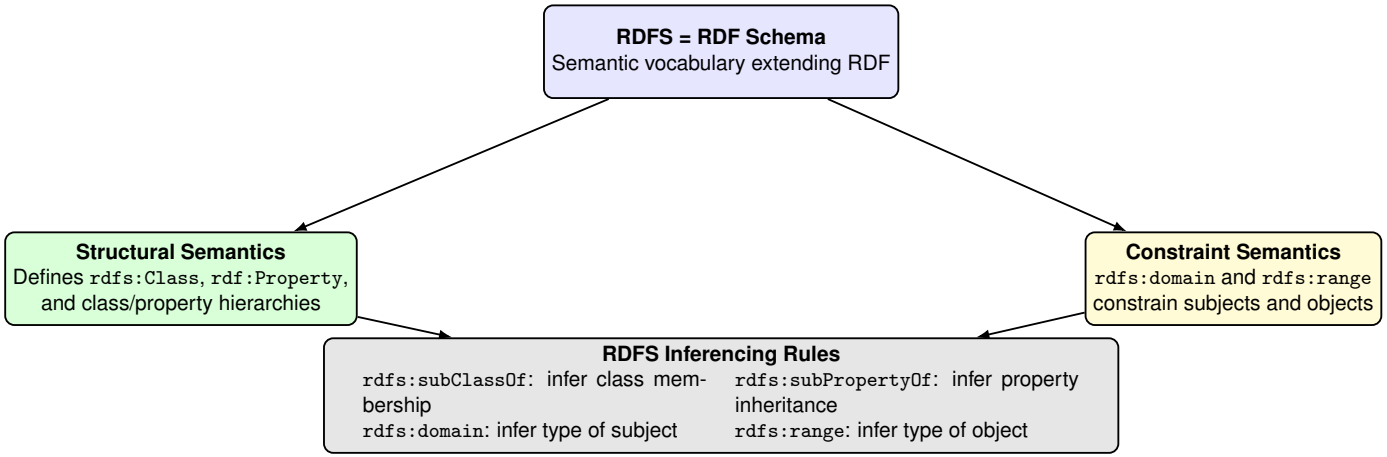
The English version of this specification is the only normative version. Non-normative translations may also be available.

Abstract

RDF Schema provides a data-modelling vocabulary for RDF data. RDF Schema is an extension of the basic RDF vocabulary.

Status of This Document

What is RDF Schema (RDFS)? (1/3)



RDFS enriches RDF with classes, constraints, and inference rules enabling semantic reasoning.

What is RDF Schema (RDFS)? (2/3)

RDFS (RDF Schema):

- is a **semantic extension** of RDF.
- It provides a framework for describing the vocabulary of RDF data.

Key Concepts:

- Classes:
  - Define groups of resources with shared properties.
  - Declared using `rdfs:Class`.
- Properties:
  - Used to describe relationships between resources.
  - Defined using `rdf:Property`.
- Hierarchies:
  - Support for class and property hierarchies with `rdfs:subClassOf` and `rdfs:subPropertyOf`.

What is RDF Schema (RDFS)? (3/2)

Additional RDFS Concepts:

- Domain and Range:
  - `rdfs:domain` specifies the class a property applies to.
  - `rdfs:range` defines the class of values a property can have.
- Types of Resources:
  - `rdf:type` is used to state that a resource is an instance of a class.
- Labels and Comments:
  - `rdfs:label` provides a human-readable name for a resource.
  - `rdfs:comment` adds a description or note about a resource.

RDFS Namespace

The prefix `rdfs:` refers to the RDF Schema vocabulary:
`http://www.w3.org/2000/01/rdf-schema#`

Common Namespaces Used in RDFS Documents

```
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix dc:   <http://purl.org/dc/elements/1.1/> .
```

Extract of the RDFS Vocabulary (in Turtle)

```
<http://www.w3.org/2000/01/rdf-schema#> a owl:Ontology ;
    dc:title "The RDF Schema vocabulary (RDFS)" .
rdfs:Resource a rdfs:Class ;
    rdfs:label "Resource" ;
    rdfs:comment "The class resource, everything." .
rdfs:Class a rdfs:Class ;
    rdfs:label "Class" ;
    rdfs:comment "The class of classes." ;
    rdfs:subClassOf rdfs:Resource .
rdfs:subClassOf a rdf:Property ;
    rdfs:label "subClassOf" ;
    rdfs:comment "The subject is a subclass of a class." ;
    rdfs:domain rdfs:Class ; rdfs:range  rdfs:Class .
```

RDFS – Main Predicates

RDFS Predicate	Intuitive Meaning
`rdfs:Class`	Defines a class (a category of things); instances belong to this class.
`rdf:Property`	Defines a type of relationship between resources (edges in the graph).
`rdfs:subClassOf`	If A `subClassOf` B, then: everything that is an instance of A is also an instance of B. ("A is a more specific class than B.")
`rdfs:subPropertyOf`	If P `subPropertyOf` Q, every pair (s, o) connected by P is also connected by Q. ("P is a more specific relation than Q.")
`rdfs:domain`	States the class of subjects a property applies to. If $s\ P\ o$, infer: s is of the domain class.
`rdfs:range`	States the class (or datatype) of objects a property can have. If $s\ P\ o$, infer: o is of the range class.
`rdfs:label`	Human-readable name for a resource ("display name").
`rdfs:comment`	Human-readable description explaining the meaning of a resource.
`rdfs:Resource`	The most general class — everything described in RDF is a Resource.

RDFS provides classes, constraints, and inheritance rules to enrich RDF with meaning.

RDFS- Definitions (1/2)

- `rdfs:Class`:
  - A set C such that $\forall x(\langle x \texttt{ rdf:type } C\rangle) \implies$ x is a resource of type C.
- `rdf:Property`:
  - A binary relation P between a subject s and an object o.
- `rdfs:domain`:
  - For a property P, $\langle C \texttt{ rdfs:domain } P\rangle$ means if $\langle s\ P\ o\rangle$ holds, then s is of type C.
- `rdfs:range`:
  - For a property P, $\langle C \texttt{ rdfs:range } P\rangle$ means if $\langle s\ P\ o\rangle$ holds, then o is of type C.

RDFS- Definitions (2/2)

- `rdfs:subClassOf`:
  - For classes C_1 and C_2, $\langle C_1$ `rdfs:subClassOf` $C_2\rangle$ implies $\forall x(\langle x$ `rdf:type` $C_1\rangle \implies \langle x$ `rdf:type` $C_2\rangle)$.
- `rdfs:subPropertyOf`:
  - For properties P_1 and P_2, P_1 `rdfs:subPropertyOf` P_2 means $\forall(s, o)(\langle s\ P1\ o\rangle \implies \langle s\ P2\ o\rangle)$.

RDFS- Definitions (2/2)

- `rdfs:subClassOf`:
  - For classes C_1 and C_2, $\langle C_1$ `rdfs:subClassOf` $C_2\rangle$ implies $\forall x(\langle x$ `rdf:type` $C_1\rangle \implies \langle x$ `rdf:type` $C_2\rangle)$.
- `rdfs:subPropertyOf`:
  - For properties P_1 and P_2, P_1 `rdfs:subPropertyOf` P_2 means $\forall(s, o)(\langle s\ P1\ o\rangle \implies \langle s\ P2\ o\rangle)$.

Explanation:

- `rdfs:subClassOf` indicates class hierarchies where C_1 inherits characteristics from C_2.
- `rdfs:subPropertyOf` defines property inheritance where P_1 inherits properties of P_2.

Steps for Modeling RDFS Graphs

1. Key Concepts

- Determine core entities.
- Choose which ones become `rdfs:Class`.

2. Properties

- Identify attributes and binary relations.
- Model them as `rdf:Property`.

3. Domains/ Ranges

- Set `rdfs:domain`
- Set `rdfs:range`

4. Hierarchies

- Class hierarchies: `rdfs:subClassOf`.
- Property hierarchies: `rdfs:subPropertyOf`.

5. Labels & Comments

- Human-friendly names (`rdfs:label`).
- Documentation of meaning (`rdfs:comment`).

Modeling in RDFS (1/2)

```
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

ex:Person   a rdfs:Class .
ex:Address  a rdfs:Class .
ex:Location a rdfs:Class .

ex:hasAddress a rdf:Property ;
    rdfs:domain ex:Person ;
    rdfs:range  ex:Address .

ex:hasLocation a rdf:Property ;
    rdfs:domain ex:Address ;
    rdfs:range  ex:Location .
```

Modeling in RDFS (1/2)

```
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .

ex:Person   a rdfs:Class .
ex:Address  a rdfs:Class .
ex:Location a rdfs:Class .

ex:hasAddress a rdf:Property ;
    rdfs:domain ex:Person ;
    rdfs:range  ex:Address .

ex:hasLocation a rdf:Property ;
    rdfs:domain ex:Address ;
    rdfs:range  ex:Location .
```

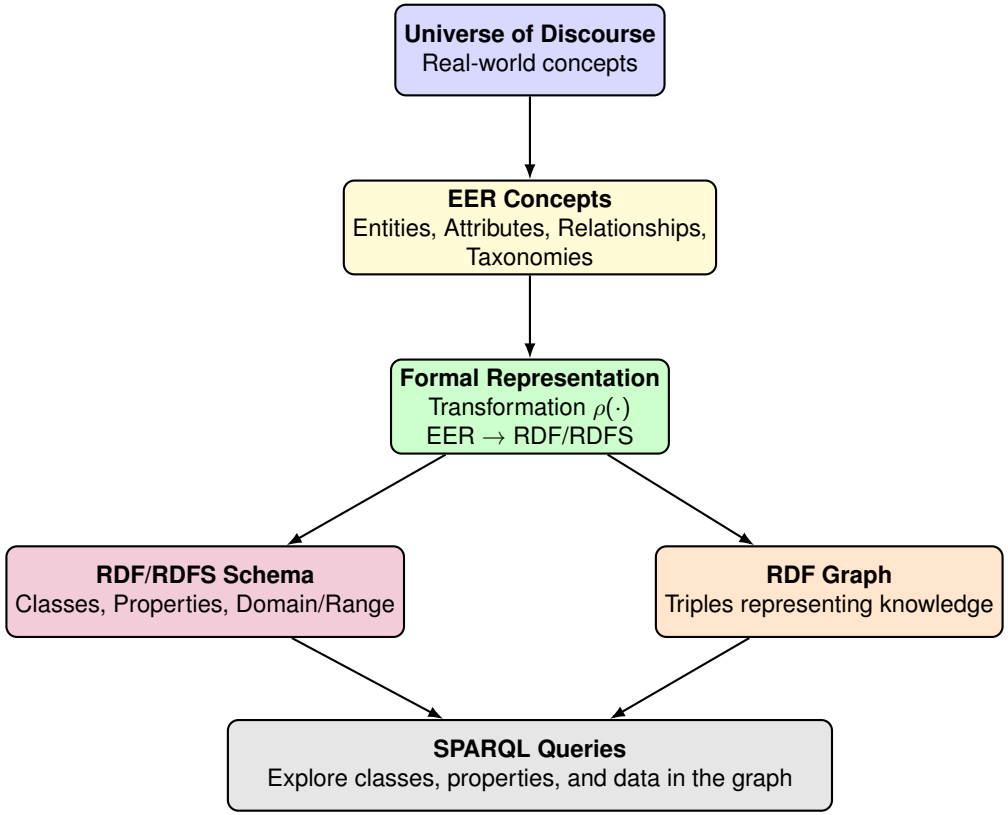
Explanation:

- `rdfs:Class` defines the entity types (e.g., `Person`, `Address`).
- `rdf:Property` creates relationships (e.g., `hasAddress`).
- `rdfs:domain` specifies the source class of the property.
- `rdfs:range` specifies the target class or data type.

Modeling in RDFS (2/2)

```
ex:street_name a rdf:Property ;
    rdfs:domain ex:Address ;
    rdfs:range xsd:string .

ex:house_number a rdf:Property ;
    rdfs:domain ex:Address ;
    rdfs:range xsd:string .

ex:zip_code a rdf:Property ;
    rdfs:domain ex:Address ;
    rdfs:range xsd:string .

ex:state a rdf:Property ;
    rdfs:domain ex:Location ;
    rdfs:range xsd:string .

ex:city a rdf:Property ;
    rdfs:domain ex:Location ;
    rdfs:range xsd:string .

ex:country a rdf:Property ;
    rdfs:domain ex:Location ;
    rdfs:range xsd:string .
```

From Conceptual to Semantic Data Modeling



From conceptual understanding $\rightarrow$ structured modeling $\rightarrow$ formal RDF schemas $\rightarrow$ usable knowledge graphs.

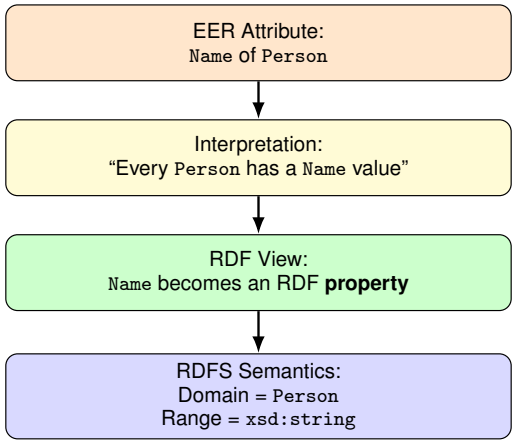
Semantic Modeling in RDFS

EER Concept	How It Is Represented in RDFS
Attributes	Become **rdf:Property**. `rdfs:domain` = class owning the attribute `rdfs:range` = datatype of values Example: `Person -name-> xsd:string`
Composite Attributes	Mapped to a new **rdfs:Class** (e.g., `Name`). Subattributes become **properties** whose domain is this new class. Example: `Name → First + Last`.
Binary Relationships	Become **rdf:Property** with: — `domain` = source class — `range` = target class If relationship has attributes → model as a class.
N-ary Relationships	Always represented as a new **rdfs:Class** modeling the n-ary relationship. Roles become **properties**, e.g., `hasCourse`, `hasStudent`.
Entity Types	Directly mapped to **rdfs:Class**. Form the backbone of the RDFS schema.
Taxonomies / Specialization	Class hierarchies are created using **rdfs:subClassOf**. Subclasses inherit all constraints from their superclass.

How Attributes Become RDF/RDFS Triples

Idea: An attribute in EER (e.g., `Name` of `Person`) becomes an RDF **property** with:

- a **domain** = the class it belongs to
- a **range** = the datatype of its value



EER attributes become RDF properties with declared domain and datatype.

EER Attributes in RDFS

EER Attribute Type	Meaning in EER	Representation in RDFS
Simple Attribute	Single atomic value (e.g., Name, Age)	One **rdf:Property** with: — `rdfs:domain = Class` — `rdfs:range = Datatype`
Multi-valued Attribute	Multiple values allowed (e.g., several Emails)	Same as simple attributes: **rdf:Property** with domain+range. Multiplicity handled at instance level.
Optional Attribute	May have no value (e.g., MiddleName)	Same as simple attribute. Optionality represented by **absence of the triple**.
Composite Attribute	Has subcomponents (e.g., Name = First + Last)	Create a new **rdfs:Class**. Each subattribute becomes **rdf:Property** with `domain = CompositeClass`.
Attributes of Relationships	Describe properties of a relationship (e.g., Grade, Date)	Relationship becomes an **rdfs:Class**. Attributes become **rdf:Property** with `domain = RelationshipClass`.

Different EER attribute types determine distinct RDF/RDFS representations.

Transforming Attributes

Definition: Let A be an attribute such that

- A is simple, multi-valued, or optional;
- D is the type of A
- A is an attribute of E, where E is an entity or relationship type.

Transforming Attributes

Definition: Let A be an attribute such that

- A is simple, multi-valued, or optional;
- D is the type of A
- A is an attribute of E, where E is an entity or relationship type.

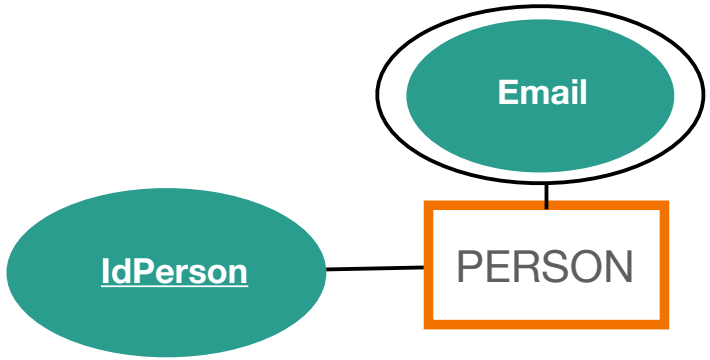
Transformation Rule:

- $\rho(A)$ is the set composed of the following triples:

```
ex:A a rdf:Property.
ex:A rdfs:domain ex:E.
ex:A rdfs:range D.
```

- `rdfs:domain` specifies the class to which the attribute applies (subject).
- `rdfs:range` specifies the data type of the value (object).

Example: Attribute Transformation




```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/xmlschema> .
@prefix ex: <http://example.org/> .

ex:PERSON a rdfs:Class .
ex:IdPerson a           rdf:Property ;
            rdfs:domain ex:PERSON ;
            rdfs:range  xsd:string .
ex:Mail     a           rdf:Property ;
            rdfs:domain ex:PERSON ;
            rdfs:range  xsd:string .
```

Transforming Binary Relationship

Definition: Let R be a binary relationship (without attributes) relating entity types $C1$ and $C2$.

Transforming Binary Relationship

Definition: Let R be a binary relationship (without attributes) relating entity types $C1$ and $C2$.
Transformation Rule:

- $\rho(R)$ is the set composed of the following triples:

```
ex:C1 a rdfs:Class .
ex:C2 a rdfs:Class .
ex:R  a rdf:Property;
      rdfs:domain ex:C1;
      rdfs:range  ex:C2.
```

Example: Binary Relationship Transformation


```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix ex: <http://example.org/> .

ex:PERSON     a rdfs:Class .
ex:UNIVERSITY a rdfs:Class .
ex:Affiliated a rdf:Property ;
              rdfs:domain ex:PERSON ;
              rdfs:range  ex:UNIVERSITY .
```

Transforming Composite Attributes

Definition: Let A be a composite attributes $A1, \ldots, An$

- $A1, \ldots, An$ are simple, multi-valued, or optional;
- $D1, \ldots, Dn$ are the types of $A1, \ldots, An$, respectively.
- A is an attribute of E, where E is an entity or relationship type.

Transforming Composite Attributes

Definition: Let A be a composite attributes $A1, \ldots, An$

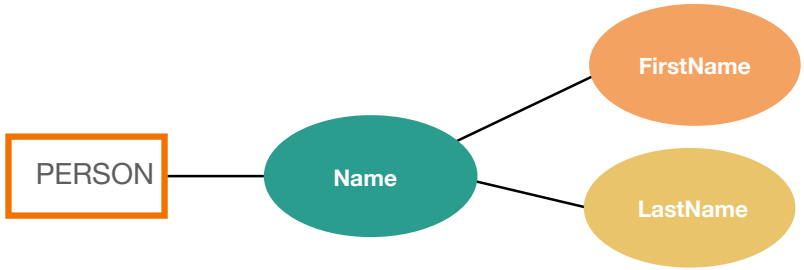
- $A1, \ldots, An$ are simple, multi-valued, or optional;
- $D1, \ldots, Dn$ are the types of $A1, \ldots, An$, respectively.
- A is an attribute of E, where E is an entity or relationship type.

Transformation Rule:

- $\rho(A)$ is the set composed of the following triples:

```
ex:A    a            rdfs:Class.
ex:hasA a            rdf:Property;
        rdfs:domain  ex:E;
        rdfs:range   ex:A.
ex:A1   a            rdf:Property;
        rdfs:domain    ex:A;
        rdfs:range     D1.
...
ex:An   a             rdf:Property;
        rdfs:domain   ex:A;
        rdfs:range    Dn.
```

Example: Composite Attribute Transformation




```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/xmlschema> .
@prefix ex: <http://example.org/> .

ex:NAME      a           rdfs:Class .
ex:PERSON    a           rdfs:Class .
ex:IdNAME    a           rdf:Property ;
             rdfs:domain ex:PERSON ;
             rdfs:range  ex:NAME .
ex:FirstName a           rdf:Property ;
             rdfs:domain ex:NAME ;
             rdfs:range  xsd:string .
ex:LastName  a           rdf:Property ;
             rdfs:domain ex:NAME ;
             rdfs:range  xsd:string .
```

Transforming Relationships (1/2)

Definition: Let R be a relationship such that R is a binary relationship between $C1$ and $C2$ but with at least one attribute A

Transforming Relationships (1/2)

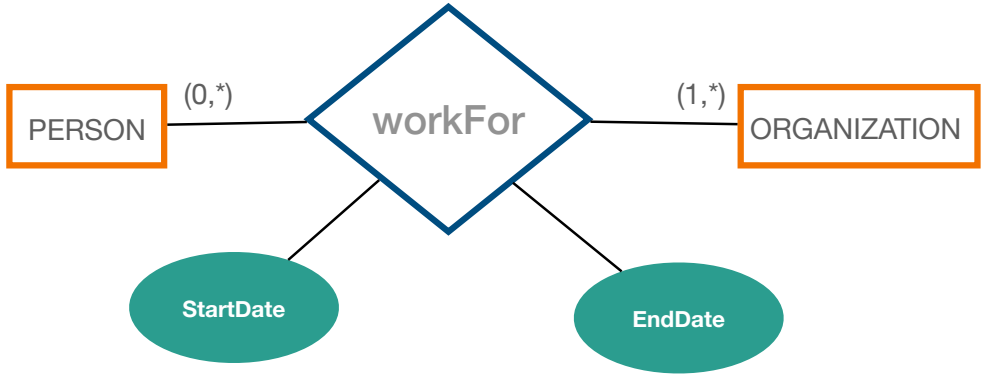
Definition: Let R be a relationship such that R is a binary relationship between $C1$ and $C2$ but with at least one attribute A

Transformation Rule:

- $\rho(R)$ is the set composed of the following triples:

```
ex:R       a    rdfs:Class.
ex:C1      a    rdfs:Class.
ex:C2      a    rdfs:Class.
ex:hasRC1  a    rdf:Property;
           rdfs:domain ex:R;
           rdfs:range  ex:C1.
ex:hasRC2  a    rdf:Property;
           rdfs:domain ex:R;
           rdfs:range  ex:C2.
```

Example- Transforming Relationships (1/2)




```
ex:WORKFOR                  a    rdfs:Class.
ex:hasWORKFORPerson         a    rdf:Property;.
                            rdfs:domain ex:WORKFOR;
                            rdfs:range  ex:PERSON.
ex:hasWORKFOROrganization   a    rdf:Property;
                            rdfs:domain ex:WORKFOR;
                            rdfs:range  ex:ORGANIZATION.
ex:StartDate                a    rdf:Property;
                            rdfs:domain ex:WORKFOR;
                            rdfs:range  xsd:date.
ex:EndtDate                 a    rdf:Property;
                            rdfs:domain ex:WORKFOR;
                            rdfs:range  xsd:date.
```

Transforming Relationships (2/2)

Definition: Let R be a relationship such that R is an n-ary relationship between $C1 \ldots Cn$

Transforming Relationships (2/2)

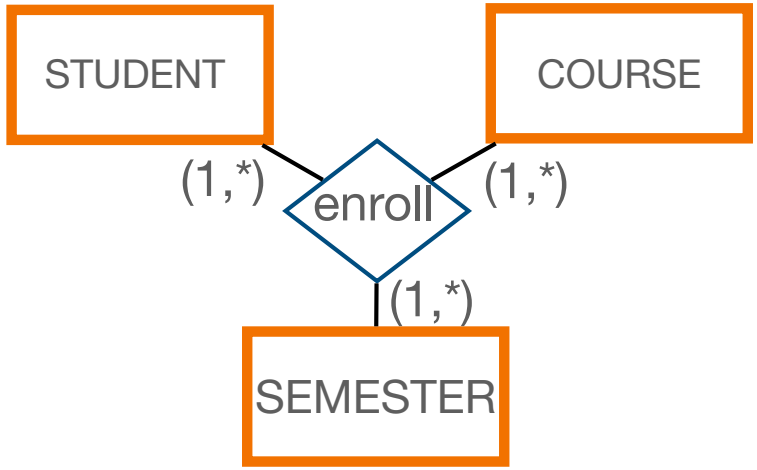
Definition: Let R be a relationship such that R is an n-ary relationship between $C1 \ldots Cn$

Transformation Rule:

- $\rho(R)$ is the set composed of the following triples:

```
ex:R      a           rdfs:Class.
ex:hasRC1 a           rdf:Property;
          rdfs:domain ex:R;
          rdfs:range ex:C1.
...
ex:hasRCn a           rdf:Property;
          rdfs:domain ex:R;
          rdfs:range  ex:Cn.
```

Example- Transforming Relationships (1/2)




```
ex:enroll          a            rdfs:Class.
ex:enrollSTUDENT   a            rdf:Property;
                   rdfs:domain  ex:enroll;
                   rdfs:range   ex:STUDENT.
ex:enrollCOURSE    a            rdf:Property;
                   rdfs:domain  ex:enroll;
                   rdfs:range   ex:COURSE.
ex:enrollSEMESTER  a            rdf:Property;
                   rdfs:domain  ex:enroll .
                   rdfs:range   ex:SEMESTER.
```

Transforming Taxonomies (1/2)

Definition: Let $C1$ and $C2$ be classes such that $C1$ is subclass of $C2$

- $\rho(C1\ \textit{subclass}\ C2)$ is the set composed of the following triples:

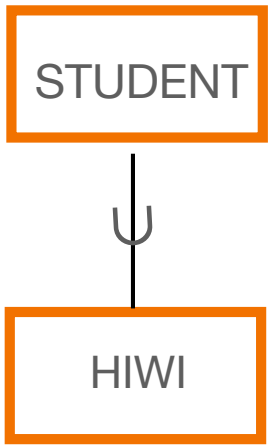
```
ex:C1  rdfs:subClassOf ex:C2.
```

Transforming Taxonomies (1/2)

Definition: Let *C*1 and *C*2 be classes such that *C*1 is subclass of *C*2

- ρ(*C*1 *subclass C*2) is the set composed of the following triples:

```
ex:C1  rdfs:subClassOf ex:C2.
```




```
ex:STUDENT a              rdfs:Class.
ex:HIWI    a              rdfs:Class.
ex:HIWI    rdfs:subClassOf ex:STUDENT.
```

Transforming Taxonomies (2/2)

Definition: Let C, $C1$, $C2$, . . . , Cn be classes such that

- $C1$, $C2$, . . . , Cn are the specialization of C.

Transforming Taxonomies (2/2)

Definition: Let C, $C1$, $C2$, ..., Cn be classes such that

- $C1$, $C2$, ..., Cn are the specialization of C.
- $\rho(C \textit{ specialized } C1, C2, \ldots, Cn)$ is the set composed of the following triples:

```
ex:C1  rdfs:subClassOf ex:C.
ex:C2  rdfs:subClassOf ex:C.
...
ex:Cn  rdfs:subClassOf ex:C.
```

Transforming Taxonomies (2/2)

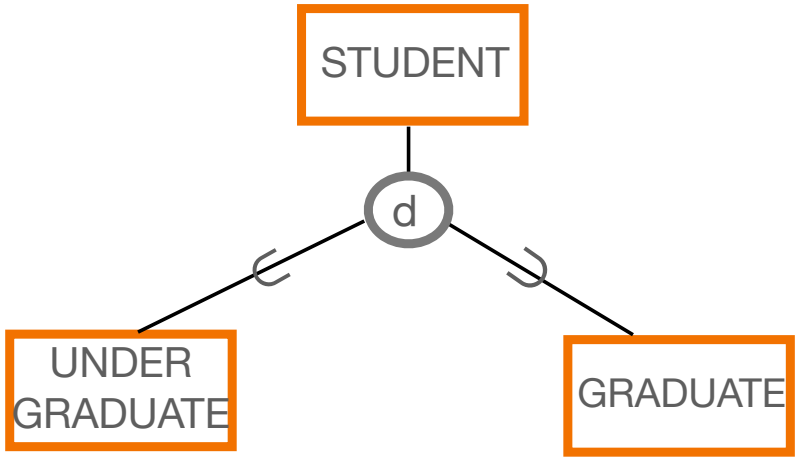
Definition: Let C, $C1$, $C2$, ..., Cn be classes such that

- $C1$, $C2$, ..., Cn are the specialization of C.
- $\rho(C \textit{ specialized } C1, C2, \ldots, Cn)$ is the set composed of the following triples:

```
ex:C1  rdfs:subClassOf ex:C.
ex:C2  rdfs:subClassOf ex:C.
...
ex:Cn  rdfs:subClassOf ex:C.
```

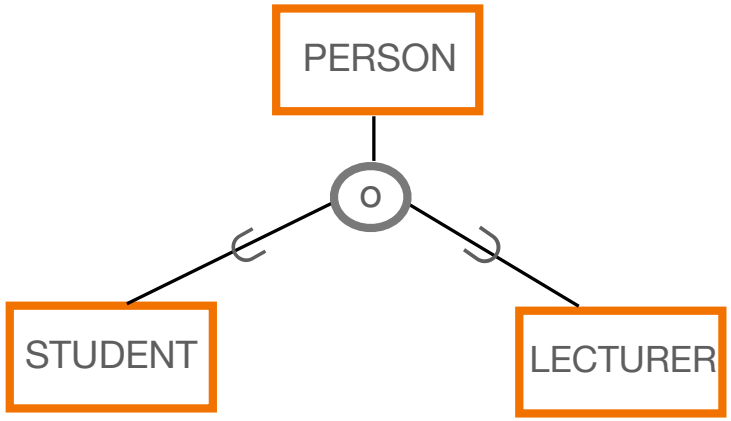
Note: Because RDFS does not provide operators to represent intersection or union, the translation ignores the type of specialization, i.e., disjoint, overlapping, total, or partial.

Example Specialization (1/2)




```
ex:STUDENT       a                rdfs:Class.
ex:UNDERGRADUATE a                rdfs:Class.
ex:GRADUATE      a                rdfs:Class.
ex:GRADUATE      rdfs:subClassOf  ex:STUDENT.
ex:UNDERGRADUATE rdfs:subClassOf  ex:STUDENT.
```

Example Specialization (2/2)




```
ex:PERSON   a rdfs:Class.
ex:STUDENT  a rdfs:Class.
ex:LECTURER a rdfs:Class.
ex:STUDENT  rdfs:subClassOf ex:PERSON.
ex:LECTURER rdfs:subClassOf ex:PERSON.
```

Benefits of Describing Schemas with RDFS

1. Expressive Representation

RDFS enables rich schema definitions with **classes** and **properties**, allowing clear modeling of structure and relationships.

2. Machine-Readable Metadata

Schemas written in RDF/RDFS are fully **machine-interpretable**, supporting automated processing, validation, and reasoning.

3. Accessible via SPARQL Endpoints

Publishing RDFS schemas through SPARQL enables **direct querying** of classes, properties, constraints, and hierarchies.

4. Schema-Driven Data Extraction

SPARQL queries can follow the **schema structure** to retrieve all instances of a class, its subclasses, or properties.

Examples of SPARQL Queries (1/3)

The following SPARQL queries can be executed over
`https://labs.tib.eu/sdm/lecture_04/sparql`

Query 1: Count how many properties each class has in the RDFS graph.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT ?class (COUNT(DISTINCT ?property) as ?numDT)
WHERE {
  ?class a rdfs:Class .
  OPTIONAL {
    ?property a rdf:Property.
    ?property rdfs:domain ?class.}}
GROUP BY ?class
ORDER BY DESC(?numDT)
```

Examples of SPARQL Queries (2/3)

Query 2: Count the number of properties with the same range class.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT ?class (COUNT(DISTINCT ?property) as ?numO)
WHERE {
  ?class a rdfs:Class .
  ?property a rdf:Property.
  ?property rdfs:range ?class.}
GROUP BY ?class
ORDER BY DESC(?numO)
```

Examples of SPARQL Queries (3/3)

Query 3: List the class(es) that have at least two properties.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT DISTINCT ?class
WHERE {
  ?class a rdfs:Class .
  ?property1 a rdf:Property.
  ?property1 rdfs:domain ?class.
  ?property2 a rdf:Property.
  ?property2 rdfs:domain ?class.
  FILTER (?property1 != ?property2)}
```

Exercise

Participate **in the survey at Stud.IP**.

Summary- Semantic Modeling

Concept	Representation in RDFS
Attributes	Various types of attributes can be modeled: simple, multi-valued, composite, and multi-valued composite. Represented as properties with a domain (class) and range (data type).
Relationships	Can be of any arity, with cardinality constraints to represent the participation of instances in related classes. Represented as properties with domains and ranges as classes. Binary relationships with attributes or arities greater than 2 should be modeled as RDF classes.
Classes	Refer to classes or entity types. Defined using the concept `rdfs:Class` in RDF Schema.
Taxonomies and Specializations	Support subclassing, classification, and overlapping classifications. Subclasses are defined using `rdfs:subClassOf` to establish hierarchical relationships between classes.

Summary – RDF/RDFS

1. Attributes & Binary Relationships

Simple, multi-valued, and optional attributes, as well as binary relationships, **become RDF properties** with appropriate `rdfs:domain` and `rdfs:range`.

2. Composite Attributes & N-ary Relationships

Complex structures are **modeled as RDFS classes**, with their components represented through RDF properties.

3. Entity Types

EER entity types **map directly to RDFS classes** that describe the core concepts of the domain.

4. Taxonomies

Hierarchies are represented with `rdfs:subClassOf`, enabling **inheritance of structure and semantics**.

5. Domain & Range

`rdfs:domain` and `rdfs:range` enforce **semantic consistency**, ensuring properties connect the right classes and datatypes.

RDF/RDFS provides a clean, formal, and machine-interpretable representation of all EER basic constructs.

References I

R. Elmasri and S. B. Navathe.
Fundamentals of Database Systems, 2nd Edition.
Benjamin/Cummings, 1994.